DEMOCRITUS UNIVERSITY OF THRACE
FACULTY OF HEALTH SCIENCES
SCHOOL OF MEDICINE

ATHENA
RESEARCH AND INNOVATION CENTER IN
INFORMATION COMMUNICATION AND KNOWLEDGE TECHNOLOGIES

Design and Evaluation of a Retrieval Augmented Artificial Intelligence Approach for Biomedical Literature Extraction

MSc THESIS

MSc in

BIOMEDICAL INFORMATICS

Nikolaos Chaikalis

Supervisor:
Eleni Kaldoudi
Professor of Medical Physics and Medical Informatics
School of Medicine, Democritus University of Thrace, Greece

ALEXANDROUPOLI, GREECE

AUGUST 2026

MSc Thesis

Submitted for the Degree of Master of Science in Biomedical Informatics,

Jointly Organized by the

School of Medicine, Democritus University of Thrace, Greece and

ATHENA Research and Innovation Center in Information Communication and Knowledge Technologies, Greece

Design and Evaluation of a Retrieval Augmented Artificial Intelligence Approach for Biomedical Literature Extraction

by

Nikolaos Chaikalis

Supervised by

Eleni Kaldoudi

Professor of Medical Physics and Medical Informatics
School of Medicine
Democritus University of Thrace, Greece

Approved on 24 August 2026 by the Examination Committee:

Eleni Kaldoudi	**Stylianos Didaskalou**	**Aliki Fiska**
Professor School of Medicine, Democritus University of Thrace, Greece	Associate Researcher ATHENA Research and Innovation, Greece	Professor School of Medicine, Democritus University of Thrace, Greece

Μεταπτυχιακή Διπλωματική Εργασία

Υποβλήθηκε για το Πτυχίο Μεταπτυχιακών Σπουδών στην Βιοϊατρική Πληροφορική

που Συνδιοργανώνεται από το

Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης και

ΑΘΗΝΑ Ερευνητικό Κέντρο Καινοτομίας στις Τεχνολογίες της Πληροφορίας, των Επικοινωνιών και της Γνώσης

Σχεδιασμός και Αξιολόγηση Συστήματος Τεχνητής Νοημοσύνης με Παραγωγή Επαυξημένη με Ανάκτηση για την Αναζήτηση Βιοϊατρικής Βιβλιογραφίας

του

Νικολάου Χαϊκάλη

Επιβλέπουσα

Ελένη Καλδούδη

Καθηγήτρια Ιατρικής Φυσικής – Ιατρικής Πληροφορικής
Τμήμα Ιατρικής
Δημοκρίτειο Πανεπιστήμιο Θράκης

Εγκρίθηκε στις 24 Αυγούστου 2026 από την Εξεταστική Επιτροπή:

Ελένη Καλδούδη	**Στυλιανός Διδασκάλου**	**Αλίκη Φίσκα**
Καθηγήτρια Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης	Συνεργαζόμενος Ερευνητής Ερευνητικό Κέντρο ΑΘΗΝΑ	Καθηγήτρια Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης

Chaikalis N., Design and Evaluation of a Retrieval Augmented Artificial Intelligence Approach for Biomedical Literature Extraction, MSc Thesis, Master of Science in Biomedical Informatics, School of Medicine, Democritus University of Thrace, and ATHENA Research Center, Alexandroupoli, Greece, August 2026 DOI: 10.5281/zenodo.21873341

The author declares that this thesis is his/her own work and to the best of his knowledge it:

- Does not breach copyright or other intellectual property rights of a third party.
- Does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
- Does not contain material which to a substantial extent has been accepted for the qualification of any other degree or diploma of a university or other institution of higher learning.
- Does not contain substantial portions of third party copyright material, including but not limited to charts, diagrams, graphs, photographs or maps, or in instances where it does the author has obtained permission to use such material and allow it to be made accessible worldwide via the Internet as part of this thesis.

During the preparation of this thesis Claude CLI, and ChatGPT CLI (June 2026) were used to assist with language refinement, content structuring, and editing support. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the final thesis.

This thesis was conducted in part within the context of European Union Horizon Europe project ThrombUS+, Grant Agreement No. 101137227. ThrombUS+ is co-funded by the European Union.

The contents of this thesis are solely the responsibility of the author. Approval of this thesis does not necessarily represent the official views of the Examination Committee, the School of Medicine, Democritus University of Thrace, or ATHENA Research and Innovation Center in Information Communication and Knowledge Technologies.

Statement of Contributions

The completion of thesis relied on important contributions from technical collaborators and domain experts:

Mr. Nickolaos Portokallidis, PhD Candidate, School of Medicine, Democritus University of Thrace, contributed in the design and technical decisions related to the proposed architecture.

Dr. Stylianos Didaskalou, Research Associate, ATHENA Research Center, contributed knowledge and guidance towards shaping the annotation workflow in Label Studio.

The following medical experts contributed annotations of medical literature records:

- Prof. Aliki Fiska, Professor of Anatomy, School of Medicine, Democritus University of Thrace
- Prof. Vasileios Papadopoulos, Assistant Professor of Anatomy, Democritus University of Thrace
- Mr. Iasonas Kalamporas, Undergraduate Student, School of Medicine, Democritus University of Thrace
- Mr. Gravriil Efthymiou, Undergraduate Student, School of Medicine, Democritus University of Thrace
- Mr. Konstantinos Fregkos, Undergraduate Student, School of Medicine, Democritus University of Thrace

Related Publications

Portokallidis N, Chaikalis N, Kalampokas I, Efthymiou G, Fregkos K, Papadopoulos V, Fiska A, Didaskalou S, Yu H, Kaldoudi E, Retrieval-Augmented Annotation Workflow for Venous Thromboembolism Risk-Factor Literature Screening. 2026 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology, 31 August – 2 September, Athens, Greece

Χαϊκάλης Ν., Σχεδιασμός και Αξιολόγηση Συστήματος Τεχνητής Νοημοσύνης με Παραγωγή Επαυξημένη με Ανάκτηση για την Αναζήτηση Βιοϊατρικής Βιβλιογραφίας, Μεταπτυχιακή Διπλωματική Εργασία, ΔΠΜΣ Βιοϊατρική Πληροφορική, Τμήμα Ιατρικής, ΔΠΘ και ΑΘΗΝΑ ΕΚ, Αλεξανδρούπολη, Αύγουστος 2026. DOI: 10.5281/zenodo.21873341

Ο/Η συγγραφέας δηλώνει ότι αυτή η διπλωματική είναι δικό του δημιούργημα και

- Δεν προσβάλλει πνευματικά δικαιώματα τρίτων.
- Δεν περιέχει υλικό που έχει δημοσιευτεί ή συνταχθεί από τρίτους, εκτός από τις περιπτώσεις που έχει γίνει κατάλληλη μνεία με χρήση πλήρους και ορθής αναφοράς στην πηγή.
- Δεν περιέχει υλικό που σε σημαντικό βαθμό έχει χρησιμοποιηθεί για άλλο πτυχίο σε άλλο Πανεπιστήμιο ή Ίδρυμα.
- Δεν περιέχει σε τμήματα από πνευματική ιδιοκτησία τρίτων, συμπεριλαμβανομένου πινάκων, διαγραμμάτων, γραφημάτων, εικόνων, φωτογραφικών, χαρτών, ή σε περίπτωση που περιέχει ο/η συγγραφέας έχει λάβει γραπτή άδεια για την χρήση αυτών των στοιχείων και την διάθεσή τους ως μέρος της διπλωματικής μέσω του διαδικτύου.

Κατά τη συγγραφή της παρούσας διατριβής χρησιμοποιήθηκαν τα εργαλεία τεχνητής νοημοσύνης Claude CLI, και ChatGPT CLI (June 2026) ως βοήθημα για βελτίωση της σύνταξης και της δομής του κειμένου και υποστήριξη της επιμέλειας. Μετά τη χρήση του εργαλείου, ο/η συγγραφέας επιμελήθηκε το περιεχόμενο και φέρει την πλήρη ευθύνη για το περιεχόμενο της τελικής εργασίας.

Η διατριβή εκπονήθηκε εν μέρει στο πλαίσιο του έργου ThrombUS+, με αριθμό σύμβασης χρηματοδότησης 101137227. Το έργο ThrombUS+ συγχρηματοδοτείται συγχρηματοδοτείται από την Ευρωπαϊκή Ένωση στο πλαίσιο του προγράμματος «Ορίζοντας Ευρώπη».

Τα περιεχόμενα αυτής της διατριβής είναι αποκλειστική ευθύνη του/της συγγραφέα. Η έγκριση της διατριβής Θράκης δεν δηλώνει απαραιτήτως την αποδοχή των απόψεων του συγγραφέα από την Εξεταστική Επιτροπή, το Τμήμα Ιατρικής του Δημοκριτείου Πανεπιστημίου, ή το ΑΘΗΝΑ Ερευνητικό Κέντρο Καινοτομίας στις Τεχνολογίες της Πληροφορίας, των Επικοινωνιών και της Γνώσης.

Δήλωση Συμβολής Τρίτων

Η ολοκλήρωση της παρούσας διατριβής βασίστηκε σε σημαντικές συνεισφορές των παρακάτω συνεργατών:

Ο κ. Νικόλαος Πορτοκαλλίδης, Υποψήφιος Διδάκτωρ του Τμήματος Ιατρικής του Δημοκριτείου Πανεπιστημίου Θράκης, συνέβαλε στον σχεδιασμό και στις τεχνικές αποφάσεις που αφορούν στην προτεινόμενη αρχιτεκτονική.

Ο Δρ. Στυλιανός Διδασκάλου, Συνεραζόμενος Ερευνητής, Ερευνητικό Κέντρο «ΑΘΗΝΑ», συνέβαλε με τις γνώσεις και την καθοδήγησή του στη διαμόρφωση της ροής εργασίας σχολιασμού στο Label Studio.

Οι παρακάτω ειδικοί του ιατρικού πεδίου συνέβαλαν στον σχολιασμό εγγραφών της ιατρικής βιβλιογραφίας:

- Αλίκη Φίσκα, Καθηγήτρια Ανατομίας, Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης
- Βασίλειος Παπαδόπουλος, Επίκουρος Καθηγητής Ανατομίας, Δημοκρίτειο Πανεπιστήμιο Θράκης
- Ιάσονας Καλαμπόρας, Προπτυχιακός Φοιτητής, Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης
- Γαβριήλ Ευθυμίου, Προπτυχιακός Φοιτητής, Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης
- Κωνσταντίνος Φράγκος, Προπτυχιακός Φοιτητής, Τμήμα Ιατρικής, Δημοκρίτειο Πανεπιστήμιο Θράκης

Σχετικές Δημοσιεύσεις

Portokallidis N, Chaikalis N, Kalampokas I, Efthymiou G, Fregkos K, Papadopoulos V, Fiska A, Didaskalou S, Yu H, Kaldoudi E, Retrieval-Augmented Annotation Workflow for Venous Thromboembolism Risk-Factor Literature Screening. 2026 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology, 31 August – 2 September, Athens, Greece

Design and Evaluation of a Retrieval Augmented Artificial Intelligence Approach for Biomedical Literature Extraction

Nikolaos Chaikalis

SUMMARY

The volume of biomedical literature indexed in PubMed exceeds the practical capacity of manual review, while standalone generative models produce fluent responses without verifiable provenance. Although biomedical literature evolves and increases continuously, clinicians and researchers primarily rely on static literature reviews and keyword based search systems. These traditional approaches lack semantic understanding, structured retrieval, automated knowledge freshness monitoring, and integrated validation workflows.

This thesis presents the design, implementation, and evaluation of a domain adapted Retrieval Augmented Generation platform tailored for extraction of records related to thrombosis risk factors from PubMed literature. The proposed system integrates semantic embedding based retrieval, vector database storage, configurable large language model generation, and a continuous DOI level ingestion scheduler. A human in the loop curation workflow is incorporated through annotation tooling, enabling validation and iterative improvement of extracted knowledge.

The platform follows modular architectural principles emphasizing reproducibility, observability, and configuration driven experimentation. A multi phase evaluation framework assesses retrieval quality, answer grounding, extraction accuracy, and operational reproducibility using qrels based ranking metrics, RAGAS metrics, extraction metrics, and recorded run artifacts. The June 2026 generation benchmark evaluates 25 thrombosis oriented questions across LLM only, RAG without HyDE, and RAG with HyDE modes, with extraction metrics reported only for the 13 questions that have derived gold factors, while the finalized June 2026 retrieval benchmark reports qrels based ranking metrics from a separate 25-query independent retrieval phase.

The thesis contributes a reproducible biomedical RAG architecture, an incremental PubMed scheduling mechanism, a human in the loop integration pipeline, and a structured evaluation methodology. The empirical findings are interpreted conservatively: the platform demonstrates auditable experimental architecture and reproducible benchmark artifacts, not clinical readiness.

Σχεδιασμός και Αξιολόγηση Συστήματος Τεχνητής Νοημοσύνης με Παραγωγή Επαυξημένη με Ανάκτηση για την Αναζήτηση Βιοϊατρικής Βιβλιογραφίας

Νικόλαος Χαϊκάλης

ΠΕΡΙΛΗΨΗ

Ο όγκος της βιοϊατρικής βιβλιογραφίας που ευρετηριάζεται στη PubMed υπερβαίνει την πρακτική δυνατότητα χειροκίνητης αναζήτησης και ανάκτησης. Ενώ η διαθέσιμη βιβλιογραφία εξελίσσεται και αυξάνει συνεχώς, οι κλινικοί ιατροί και οι ερευνητές βασίζονται κυρίως σε στατικές βιβλιογραφικές ανασκοπήσεις και σε συστήματα αναζήτησης βασισμένα σε λέξεις-κλειδιά. Οι παραδοσιακές αυτές προσεγγίσεις στερούνται σημασιολογικής κατανόησης, δομημένης ανάκτησης, αυτοματοποιημένης παρακολούθησης της επικαιρότητας της γνώσης και ενσωματωμένων ροών επικύρωσης.

Η παρούσα διπλωματική εργασία παρουσιάζει τη σχεδίαση, την υλοποίηση και την αξιολόγηση μιας εξειδικευμένης για το πεδίο πλατφόρμας Ανάκτησης-Επαυξημένης Παραγωγής(Retrieval Augmented Generation), προσαρμοσμένης στην εξαγωγή παραγόντων κινδύνου θρόμβωσης από τη βιβλιογραφία του PubMed. Το προτεινόμενο σύστημα ενσωματώνει σημασιολογική ανάκτηση βασισμένη σε ενσωματώσεις (embeddings), αποθήκευση σε διανυσματική βάση δεδομένων, παραμετροποιήσιμη παραγωγή κειμένου από μεγάλο γλωσσικό μοντέλο και έναν συνεχή χρονοπρογραμματιστή εισαγωγής δεδομένων σε επίπεδο DOI. Μια ροή επιμέλειας με τον άνθρωπο στον βρόχο ενσωματώνεται μέσω εργαλείων επισημείωσης, επιτρέποντας την επικύρωση και τη σταδιακή βελτίωση της εξαγόμενης γνώσης.

Η πλατφόρμα ακολουθεί αρθρωτές αρχιτεκτονικές αρχές που δίνουν έμφαση στην αναπαραγωγιμότητα, στην παρατηρησιμότητα και στον πειραματισμό καθοδηγούμενο από τη διαμόρφωση. Ένα πλαίσιο αξιολόγησης πολλαπλών φάσεων εκτιμά την ποιότητα ανάκτησης, τη θεμελίωση των απαντήσεων στις πηγές, την ακρίβεια εξαγωγής και τη λειτουργική αναπαραγωγιμότητα, χρησιμοποιώντας μετρικές κατάταξης βασισμένες σε qrels, μετρικές RAGAS, μετρικές εξαγωγής και καταγεγραμμένα τεκμήρια εκτελέσεων. Το σημείο αναφοράς παραγωγής του Ιουνίου 2026 αξιολογεί 25 ερωτήσεις προσανατολισμένες στη θρόμβωση σε τρεις λειτουργίες, μόνο LLM, RAG χωρίς HyDE και RAG με HyDE, με τις μετρικές εξαγωγής να αναφέρονται μόνο για τις 13 ερωτήσεις που διαθέτουν παραγόμενους παράγοντες αναφοράς, ενώ το οριστικοποιημένο σημείο αναφοράς ανάκτησης του Ιουνίου 2026 αναφέρει μετρικές κατάταξης βασισμένες σε qrels από μια χωριστή, ανεξάρτητη φάση ανάκτησης 25 ερωτημάτων.

Η διπλωματική συνεισφέρει μια αναπαραγώγιμη βιοϊατρική αρχιτεκτονική RAG, έναν μηχανισμό σταδιακού χρονοπρογραμματισμού του PubMed, μια ροή ενσωμάτωσης με τον άνθρωπο στον βρόχο και μια δομημένη μεθοδολογία αξιολόγησης. Τα εμπειρικά ευρήματα ερμηνεύονται συντηρητικά: η πλατφόρμα καταδεικνύει μια ελέγξιμη πειραματική αρχιτεκτονική και αξιολόγηση.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Professor Eleni Kaldoudi, for her guidance, academic insight, and continuous support throughout the development of this thesis.

I also thank the supervising PhD candidate, Nickolaos Portokalidis, for his guidance, constructive feedback, and technical discussions during all phases of the thesis.

I am grateful to the faculty and colleagues of the MSc Biomedical Informatics program for fostering an interdisciplinary academic environment.

Finally, I would like to thank my family and close friends for their continuous support and encouragement. I would also like to thank my dog for patiently tolerating the postponed walks and for providing moments of balance and perspective throughout this journey.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

Acronym	Description
API	Application Programming Interface
ARES	Automated RAG Evaluation System
BioASQ	Biomedical Semantic Indexing and Question Answering
BioBERT	Biomedical Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DALY	Disability Adjusted Life Years
DOI	Digital Object Identifier
HTTP	Hypertext Transfer Protocol
HyDE	Hypothetical Document Embeddings
LLM	Large Language Model
MedCPT	Medical Contrastive Pre-trained Transformer
MMR	Mean Reciprocal Rank
NCBI	National Center for Biotechnology Information
nDCG	Normalized Discounted Cumulative Gain
NIH	National Institutes of Health
NLP	Natural Language Processing
NML	National Library of Medicine
PMID	PubMed Identifier
qrels	Query Relevance Labels
RAG	Retrieval Augmented Generation

Acronym	Description
RAGAS	Retrieval Augmented Generation Assessment
SDK	Software Development Kit
TREC	Text Retrieval Conference
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
YAML	Yet Another Markup Language

Chapter 1| Introduction

1.1. Motivation

The volume of biomedical literature indexed in repositories such as PubMed has grown to a scale that exceeds the practical capacity of manual review, even within narrowly defined clinical domains [1]. Existing retrieval tools only partially address this challenge. Keyword-based search interfaces remain effective for broad bibliographic exploration, but their performance degrades when terminology is heterogeneous, evolving, or context-dependent [2]. Even when relevant records are successfully retrieved, the burden of synthesizing evidence across multiple heterogeneous sources into an application-ready conclusion remains largely unresolved by search infrastructure alone.

At the same time, standalone artificial intelligence (AI) generative methods introduce a second limitation. A generative model can produce fluent and coherent responses while failing to provide adequately verifiable evidence. For general-purpose tasks this may be acceptable in exploratory settings, but for biomedical decision support it is not. Biomedical workflows require justifiable outputs, explicit provenance, and reproducible behavior under controlled conditions. Consequently, retrieval quality and content generation quality cannot be evaluated independently if the system is intended for evidence-grounded use.

Systems that couple automated literature acquisition with Retrieval Augmented Generation (RAG) offer a candidate response to this problem [3]. In this paradigm, the model response is conditioned on context retrieved at query time rather than relying only on parametric memory. This creates a stronger basis for verifiability because candidate evidence can be examined and linked to answer content. The research presented here concerns the design, implementation, and evaluation of a biomedical RAG platform. Thrombosis risk factor identification is the use case selected to demonstrate and evaluate the platform.

1.2. Problem statement

The research addresses the integrated problem of continuous knowledge acquisition, structured indexing, and reproducible evaluation for biomedical question answering supported by retrieval augmentation. Each stage can be implemented in isolation, yet isolated optimization frequently produces systems that are difficult to validate, hard to compare, and fragile under operational change.

The first dimension of the problem concerns knowledge freshness. Biomedical evidence changes continuously, and any static corpus progressively diverges from current literature. A platform that cannot update systematically will eventually produce answers based on stale context, even when retrieval and generation components are individually strong.

The second dimension concerns representational suitability for retrieval. New documents must be transformed into indexable units that preserve enough semantic signal for meaningful nearest-neighbor retrieval while remaining computationally tractable. This requires decisions about document granularity, metadata retention, and update semantics. Inadequate representation introduces hidden bias into retrieval, which then propagates into synthesis quality.

The third dimension concerns empirical validity. A system that produces compelling demonstrations but lacks reproducible evaluation procedures cannot support rigorous comparative analysis. Retrieval and generation behavior must be measured through repeatable experimental stages with preserved intermediate files, so

that observed performance can be interpreted in relation to the system configuration rather than uncontrolled execution.

Existing approaches commonly privilege one dimension at the expense of the others. Static retrieval systems tend to support stable indexing but offer limited mechanisms for continuous acquisition. Standalone generative systems can deliver fluent responses but weaken evidence accountability when retrieval grounding is absent or inconsistent. Ad hoc evaluation scripts may provide snapshots of performance but often fail to preserve sufficient process traceability for robust replication. The central research problem is therefore architectural: how to design a biomedical platform that unifies continuous ingestion, retrieval-grounded synthesis, and reproducible evaluation without collapsing these requirements into disconnected tooling.

1.3. Thrombosis risk factor identification as the evaluation use case

The proposed architecture is demonstrated and evaluated in the use case of literature-based identification of evidence for thrombosis risk factors. Thrombosis is defined as the pathological formation of a blood clot within the arterial or venous vasculature, limiting normal blood flow and potentially disrupting downstream perfusion [4]. Globally, thromboembolic conditions represent a major public health burden: thrombosis is a common pathology underlying ischemic heart disease, ischemic stroke, and venous thromboembolism, and these thrombosis-related conditions account for approximately one in four deaths worldwide [5],[6]. Beyond immediate mortality, the chronic burden of thrombosis is profound. Venous thromboembolism associated with hospitalization has been reported as a leading driver of disability-adjusted life-years (DALYs) lost in low- and middle-income countries and the second-leading driver in high-income countries among selected hospital-associated adverse events [5]. It also imposes substantial healthcare costs through acute treatment, recurrent events, anticoagulation-related adverse events, and chronic complications such as post-thrombotic syndrome and chronic thromboembolic pulmonary hypertension ([7],[8].

This use case was selected because thrombosis is not a condition confined to a single medical specialty, but rather a cross-cutting complication that arises across markedly different clinical contexts, including cancer-associated thrombosis, postoperative and thromboprophylaxis settings, pregnancy and the peripartum period, obesity, and prolonged immobilization [9]-[14]. Each of these contexts is typically studied within its own clinical and research community, using domain-specific terminology, publication venues, and risk-factor framings, even though the underlying pathophysiological question, namely which conditions predispose a patient to thrombosis, is shared across all of them.

This cross-disciplinary distribution has a direct methodological consequence for literature retrieval. A clinician or researcher investigating thrombosis risk comprehensively cannot rely on a single subspecialty literature or a narrow keyword set, since relevant evidence may be reported in oncology, obstetrics, surgery, or critical care literature using discipline-specific vocabulary rather than a shared thrombosis-specific nomenclature. This reinforces the terminological heterogeneity challenge discussed in Chapter 2 and makes thrombosis risk factor identification a demanding and representative test case for a retrieval system intended to operate across heterogeneous biomedical sources rather than within a single well-delimited clinical subfield.

1.4. Research objectives

The first objective is to design a modular biomedical RAG architecture that separates acquisition, retrieval, synthesis, and evaluation concerns while preserving explicit integration boundaries. Modularity is treated as

a methodological objective rather than only a software engineering preference. It enables controlled replacement of components, supports clearer attribution of observed behavior, and reduces confounding effects during comparative experimentation.

The second objective is to enable automated literature ingestion from PubMed through scheduled execution. This objective includes systematic handling of document identity through Digital Object Identifier (DOI) and PubMed Identifier (PMID) workflows, because identifier consistency directly affects deduplication, update tracking, and downstream reproducibility.

The third objective is to define a reproducible evaluation framework for both retrieval and generation behavior. This objective requires stage-level output files, explicit run organization, and metrics-oriented execution paths that allow reanalysis. The intent is to ensure that reported behavior can be interpreted through a transparent process history rather than isolated endpoint measurements.

The fourth objective is to support evidence-grounded question answering with configurable retrieval strategies, including Hypothetical Document Embeddings (HyDE) [15], while preserving observability of system behavior. This objective introduces a deliberate tradeoff: advanced retrieval augmentation can improve semantic recall in difficult queries, yet it may increase latency variance and dependence on additional generation steps. Configurability and measurement are therefore treated as inseparable.

Chapter 2| Background and Related Work

2.1. Biomedical information retrieval

PubMed is a free, searchable database of more than 40 million citations and abstracts for biomedical and life sciences literature. It was developed and is maintained by the National Center for Biotechnology Information (NCBI) at the U.S. National Library of Medicine (NLM), which is part of the National Institutes of Health (NIH), and it remains the leading literature search tool for health professionals, researchers, and the general public.

Biomedical information retrieval also has a long benchmark tradition. BioASQ framed large-scale biomedical semantic indexing and question answering as a combined retrieval and synthesis problem over biomedical literature, with expert-created questions, retrieved snippets, and ideal answers [16]. The TREC Precision Medicine Track similarly emphasized retrieval for clinically structured cases, where evidence ranking must account for disease, gene, variant, and treatment relevance rather than superficial textual overlap [17]. These benchmark traditions justify evaluating retrieval as its own stage. If relevant evidence is absent from the retrieved context, no downstream generator can reliably recover the missing source.

Recent biomedical retrieval work has moved beyond purely lexical matching. MedCPT trains contrastive biomedical retrievers and rerankers from large-scale PubMed search logs, showing that user-query and click behavior can support zero-shot biomedical information retrieval [18]. BioBERT and related biomedical language models demonstrate that domain-specific pretraining improves biomedical text understanding compared with general-domain language representations [19]. Sentence-transformer architectures provide a general dense-embedding basis for semantic similarity search [20]. These studies also show that the embedding model, document chunking, and ranking method are empirical variables that can materially affect retrieval quality. Biomedical retrieval evaluation must therefore state these choices and distinguish their effects from downstream answer generation.

2.2. Retrieval-augmented generation (RAG)

Retrieval-Augmented Generation (RAG) couples a parametric language model with retrieved non-parametric evidence at inference time. Lewis et al. introduced this paradigm for knowledge-intensive natural language processing tasks, arguing that retrieved passages can improve factual specificity and provide an updateable memory external to the model parameters [3]. This distinction is central for biomedical work. Biomedical knowledge changes continuously and relying only on a model's parametric memory would make the system dependent on opaque training data and a fixed knowledge cutoff.

In a biomedical setting, RAG functions primarily as an information-delivery method that grounds a model in existing literature rather than assuring clinical correctness. Providing source documents can reduce some unsupported generation, but it cannot prevent inaccurate outputs. The retrieval stage selects candidate evidence from an indexed corpus, the synthesis stage generates an answer from the selected context, and citation handling links answer content to retrieved sources. This pattern reflects the literature's core motivation for RAG: explicit access to external evidence and clearer provenance than closed-book generation.

Several medical RAG systems illustrate the same direction. Clinfo.ai uses scientific literature retrieval to answer medical questions and explicitly evaluates retrieval and synthesis behavior [21]. Almanac applies retrieval-augmented language modeling to clinical medicine and includes physician evaluation over clinical

scenarios [22]. Benchmarking Retrieval-Augmented Generation for Medicine introduces MIRAGE and the MedRAG toolkit to compare corpora, retrievers, and models across medical question answering datasets [23]. Across these systems, retrieval quality, generation quality, and end-task performance are treated as distinct dimensions requiring separate evaluation, rather than being inferred from generated-answer quality alone.

Literature also motivates configurable retrieval expansion. Hypothetical Document Embeddings (HyDE) generate a synthetic answer-like passage from the query and embed that generated text to retrieve semantically related documents without requiring relevance labels [15]. This approach reframes retrieval as a generation-then-embedding problem, under the assumption that a plausible hypothetical answer shares more semantic structure with relevant documents than the original query does in isolation. Its expected benefit is improved recall for underspecified or loosely phrased queries, where lexical or dense query-side representations may fail to surface semantically relevant evidence. Its principal risk is expansion noise: when the original query is already precise, the generated passage may introduce spurious content that shifts retrieval away from the most relevant documents rather than toward them. This tradeoff positions HyDE as a configurable retrieval strategy whose benefit is query-dependent, rather than as a universally beneficial enhancement.

2.3. Biomedical RAG challenges

Biomedical RAG inherits the difficulties of biomedical information retrieval and adds generation-specific risks. The first challenge is terminology. Biomedical questions often mix clinical language, mechanistic terminology, abbreviations, disease names, procedures, and outcome measures. For example, thrombosis-oriented questions may refer to venous thromboembolism, deep vein thrombosis, pulmonary embolism, catheter failure, anticoagulation, or postoperative risk using different textual forms. Dense retrieval can reduce exact-term dependence, but it can also retrieve semantically nearby yet clinically misaligned evidence. This makes source metadata and post-retrieval evaluation essential.

The second challenge is corpus currency. PubMed-based systems operate over a literature stream that changes continuously. Static corpora are easier to reproduce, but they progressively diverge from the current state of the literature; continuously updated corpora better preserve currency, but they require identifier tracking, deduplication, and run-level traceability. The literature on RAG in healthcare identifies up-to-date external evidence as one of the main motivations for RAG, while also emphasizing that healthcare RAG still lacks standardized evaluation and governance practices [24].

The third challenge is grounding. A generated answer can be fluent, relevant, and still unsupported by the retrieved sources. Conversely, a response can be faithful to retrieved context that is incomplete, outdated, or not clinically authoritative. This distinction is especially important in medicine. Evaluations of medical RAG systems show improvements from retrieval in some clinical tasks but also conclude that RAG-supported outputs remain insufficient for autonomous clinical use [25]. This distinction between fluency, faithfulness, and clinical authority underscores that retrieval grounding alone does not guarantee clinical validity, particularly when retrieved context is itself incomplete or outdated.

The fourth challenge is corpus granularity. Biomedical articles contain titles, abstracts, methods, results, discussion sections, and bibliographic metadata. Retrieval over entire documents can hide relevant passages, while retrieval over very small fragments can lose context. Passage- and snippet-level retrieval, as used in

BioASQ and related systems, addresses this trade-off by retrieving focused text while retaining links to the source publication. The appropriate granularity remains dependent on the corpus and evaluation setting.

2.4. Evaluation of RAG systems

Evaluating RAG systems requires isolating distinct failure modes, because retrieval errors, grounding errors, and task-output errors arise from fundamentally different algorithmic bottlenecks. Retrieval metrics such as normalized discounted cumulative gain, precision at k, recall at k, and mean reciprocal rank measure whether relevant sources are ranked highly enough to enter the answer context. Normalized discounted cumulative gain is particularly appropriate when relevance is graded because it rewards ranking more useful evidence earlier [26]. Benchmarking information retrieval (BEIR), the standard evaluation suite for information retrieval models, reinforces the need to test retrieval systems across heterogeneous tasks rather than relying on a single narrow benchmark [27]. In the biomedical domain, BioASQ and PubMedQA further show that biomedical question answering requires evidence selection and answer evaluation over biomedical abstracts and expert-derived labels [16],[28].

Answer-level evaluation adds a separate layer. RAGAS proposes reference-free metrics for RAG pipelines, including faithfulness, answer relevancy, and context precision [29]. Answer relevancy assesses whether the response addresses the question, faithfulness estimates whether answer claims are supported by retrieved contexts, and context precision estimates whether retrieved contexts are useful for the answer. These metrics should not be interpreted as biomedical truth verification. They diagnose grounding behavior relative to supplied context.

Other RAG evaluation frameworks support this decomposition. ARES (Automated RAG Evaluation System) evaluates context relevance, answer faithfulness, and answer relevance using synthetic data and smaller human-labeled validation sets [30]. RAGChecker argues for fine-grained diagnosis of retrieval and generation modules instead of a single aggregate RAG score [31]. Both approaches show the value of reporting retrieval and generation separately. They also leave unresolved how automated scores should be calibrated for biomedical validity.

For thrombosis-oriented questions, task-specific extraction measures are also relevant. A response may be grounded and relevant but still miss a risk factor or add one unsupported by the reference set. Precision, recall, and F1 over a defined vocabulary can measure this behaviour. This aligns with biomedical natural language processing (NLP) practice, where information extraction is evaluated against curated labels rather than only through free-form assessment.

2.5. Human validation and annotation

Human validation is a methodological requirement in biomedical NLP because gold-standard labels are rarely self-evident. Deleger et al. describe gold-standard corpus construction for medical NLP using double annotation, consensus, and inter-annotator agreement measurement [32]. The i2b2/UTHealth risk-factor shared task illustrates this approach concretely: two independent annotators reviewed each clinical record, conflicting assessments were resolved through a formal human adjudication process, and detailed annotation guidelines governed how risk factors were labeled [33],[34]. This collaborative workflow established a reliable ground-truth dataset that can serve as a trusted baseline for evaluating automated systems, demonstrating how rigorous expert annotation schemas support reproducible evaluation in clinical information extraction.

Human-in-the-loop workflows can improve annotation efficiency without removing human responsibility. Neveol et al. found that semi-automatic semantic annotation of PubMed queries reduced manual effort while preserving or improving annotation quality [35]. Software can prepare candidate records, normalize metadata, and structure exports, while human review remains responsible for judging label meaning and evidence adequacy.

The strength of a human-reviewed dataset depends on its annotation guidelines, reviewer roles, agreement assessment, and adjudication process. A software interface can preserve DOI, PMID, title, abstract, and annotation fields, but those technical records do not by themselves establish a fully adjudicated clinical gold standard. Literature curation and extraction evaluation must therefore be distinguished from prospective clinical safety, clinician-workflow benefit, or treatment validity.

2.6. Local LLM deployment considerations

The local deployment of the large language model (LLM) is relevant to biomedical RAG for privacy, reproducibility, and operational control. Healthcare studies have explored local or privacy-preserving LLM pipelines for clinical text review and information retrieval. Vaid et al. used a local open-source LLM with retrieval over historic echocardiogram reports, showing that local deployment can support clinician-facing retrieval tasks while still producing errors that require validation [36]. Wiest et al. similarly evaluated privacy-preserving LLM use for structured medical information retrieval, demonstrating the feasibility of local clinical text extraction while comparing model outputs against expert-derived references [37].

Local deployment does not by itself make a system safe. Reviews of LLMs in medicine emphasize hallucination, bias, validation, and governance risks [38]. Open medical LLM deployment also requires transparent model provenance, version control, and auditable update processes [39].

Local LLM deployment cannot be established from the model name alone. A reproducible run should record the model family, model tag or digest, quantization, context length, temperature, seed where supported, prompt template, embedding model, vector-index state, and corpus snapshot. The Llama 3 model report describes the model family and its release context [40]. However, quantized model files used for local inference may differ from the original model configuration. Local execution can reduce third-party data transfer, but it does not eliminate privacy risks, as language models may memorize and reproduce training data under certain conditions [41].

2.7. Research gaps and positioning of the present work

The reviewed literature establishes mature foundations for biomedical retrieval, RAG, biomedical question-answering evaluation, annotation workflows, and local LLM deployment. The gap addressed by the present work is not the invention of a new biomedical language model or a new clinical benchmark. It is the integration problem: how to combine continuous PubMed-oriented acquisition, chunked vector indexing, evidence-grounded synthesis, human curation, and reproducible evaluation in one research platform.

Existing biomedical IR benchmarks motivate rigorous retrieval evaluation, but they do not by themselves define an operational ingestion architecture. Medical RAG systems demonstrate the value of retrieved evidence, but they often differ in corpora, retrievers, models, and evaluation protocols, making direct comparison difficult. RAG evaluation frameworks provide useful grounding diagnostics, but they do not verify

biomedical correctness. Human validation workflows create more credible reference data, but they require explicit schema design and annotation governance. Local LLM deployment improves control, but it introduces model-version and resource constraints.

The present work is positioned at the intersection of these gaps. Its contribution is an evidence-aware biomedical RAG platform and evaluation workflow for medical literature questions, demonstrated through the use case of identifying literature that reports risk factors related to thrombosis. The platform is modular: the biomedical RAG service handles ingestion, retrieval, and synthesis; the scheduler handles continuous literature acquisition; the evaluation subsystem performs phase-based measurement; and the human curation workflow supplies reviewed reference records. The study evaluates a concrete system configuration under documented constraints rather than claiming general clinical validity or universal superiority over other biomedical RAG architectures.

Chapter 3| Methodology

3.1. System Architecture

3.1.1. Requirements and scope

The scope of the proposed system is not merely to enable retrieval of the relevant literature, but to couple retrieval quality with lifecycle control, operational transparency, and experiment reproducibility. For this reason, the system is scoped as an integrated architecture rather than a single question answering service.

The dominant architectural driver is reproducibility. Explicit job states, stored run records, and staged evaluation files record how each result was produced. This requirement motivates asynchronous job orchestration and storage of both operational and experimental outputs.

A second driver is traceability across subsystem boundaries. Scheduler decisions, ingestion outcomes, and retrieval outputs must remain linkable so that failure analysis and evaluation interpretation can be performed without implicit assumptions. The architecture therefore favors explicit state transitions and structured interfaces over opaque internal coupling.

External dependency constraints are also central. The platform depends on remote biomedical data services, vector infrastructure, and generation infrastructure; consequently, readiness semantics, timeout handling, and controlled retry behavior are architectural requirements rather than implementation details. The tradeoff is additional operational complexity in exchange for bounded failure behavior.

Queue capacity and worker parallelism are additional constraints that influence ingestion throughput and failure handling semantics. The architecture therefore includes explicit backpressure behavior and retry-after signaling assumptions, so that ingestion acceptance remains predictable under bursty workloads. This choice constrains unbounded resource growth and supports operational stability, while making ingestion completion latency variable under load.

3.1.2. High level architecture

The platform is organized as three cooperating subsystems with explicit interfaces. First, the Biomedical Retrieval-Augmented Generation (RAG) service exposes ingestion, retrieval, synthesis, and document management capabilities through an Application Programming Interface (API) [3]. Second, the PubMed scheduler service governs periodic and manual acquisition workflows, adding new evidence to the retrieval corpus independently of user queries. Third, the evaluation subsystem queries the API and writes the outputs of retrieval pooling, answer generation, automated quality scoring, and extraction scoring.

Figure 1 summarizes this decomposition and highlights a key design decision: scheduler execution and query-serving execution are operationally decoupled. This limits failure propagation across subsystems but introduces eventual consistency between newly acquired literature and immediately retrievable indexed content.

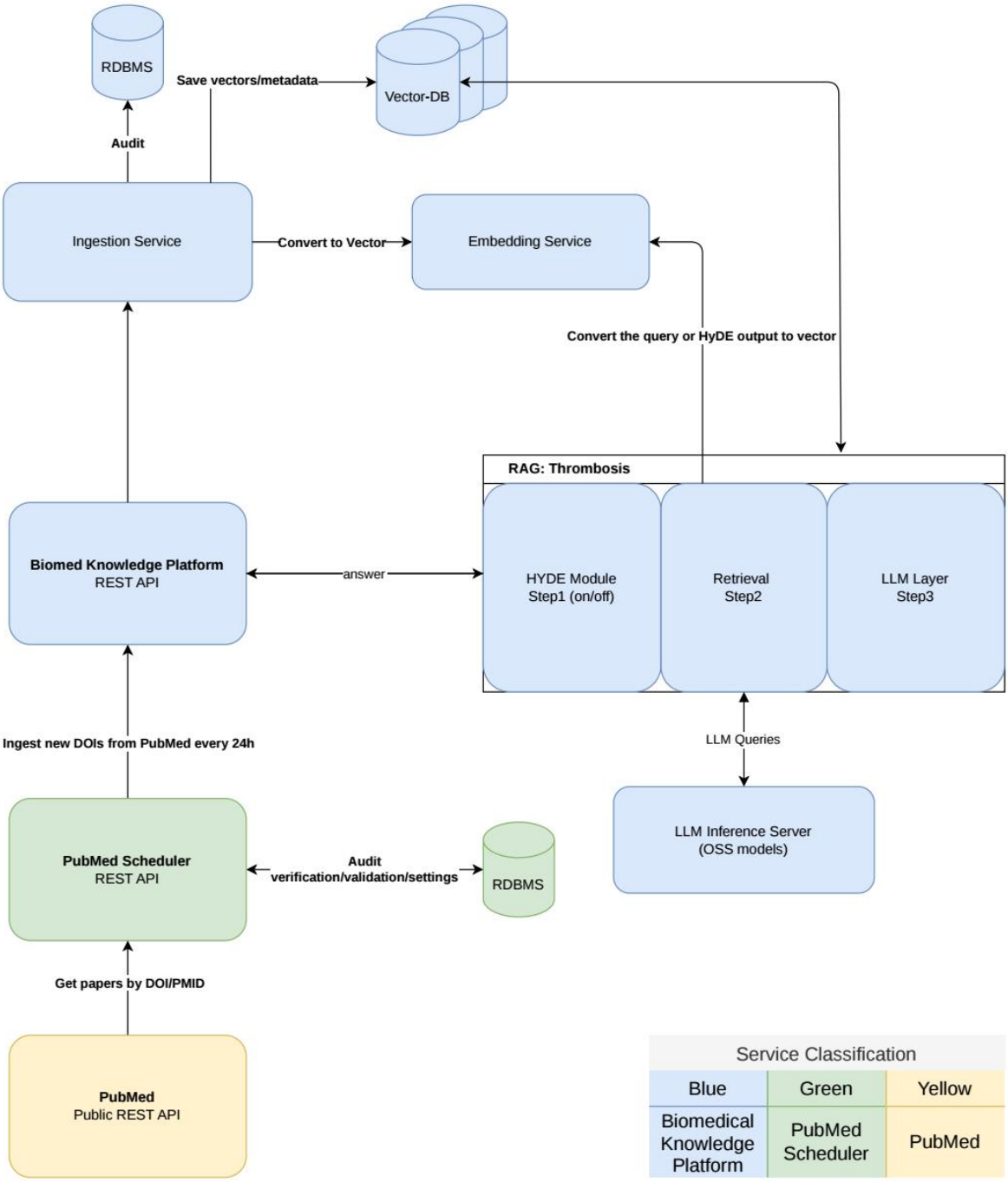

Figure 1. High level architecture of the platform.

This organization establishes both a full path from literature acquisition, to indexing and retrieval, to measured experimental outputs, and a separation between online inference concerns and experimental concerns. The biomedical RAG service is optimized for serving bounded-latency retrieval and synthesis requests, whereas the evaluation subsystem is designed to run multi-stage analyses that may be computationally intensive and asynchronous.

The biomedical RAG service and scheduler service are implemented as independent API applications with their own application and persistence lifecycles, the evaluation subsystem is implemented as a phase-oriented command-line pipeline, and the LLM server stack is separated as deployment infrastructure for model hosting. This organization expresses a deliberate architectural approach in which serving, orchestration, evaluation, and model-hosting responsibilities remain distinct while interoperating through explicit API contracts.

Subsystems can evolve at different rates without forcing tightly coupled release behavior, while shared evaluation contracts preserve comparability across runs. In practice, this supports maintainability by localizing change impact and supports reproducibility by preserving stable boundaries between acquisition, retrieval, and evaluation concerns.

3.1.3. Component architecture

The biomedical RAG service is organized around four conceptual components: ingestion, retrieval, synthesis, and document management. The ingestion component accepts normalized document payloads, enforces duplicate constraints, and orchestrates asynchronous indexing. The retrieval component transforms user questions into vector-space candidates and assembles context-bearing evidence. The synthesis component produces structured answers with explicit citations. The document management component provides lifecycle controls required for lookup, verification, and deletion.

Architecturally, the service applies Clean Architecture [42] with a ports-and-adapters realization [43]. The API layer performs transport and schema duties, use cases orchestrate business workflows, the domain layer holds framework-independent models, and adapters implement infrastructure integrations behind abstract ports. Dependency direction is inward: outer layers depend on inner policies through interfaces, while the domain remains independent of frameworks and vendors. This structure improves maintainability through low coupling and supports reproducibility by making workflow explicit at use-case boundaries.

3.1.4. Scheduler architecture and continuous literature acquisition

The scheduler service exists to convert irregular manual updates into controlled continuous literature acquisition. Its architecture persists in query definitions, creates run records, and stores per-query and per-document outcomes to maintain historical accountability of acquisition behavior.

Execution is intentionally asynchronous. Trigger operations start runs rapidly, while background tasks perform PubMed search, incremental filtering, duplicate checks against existing indexed documents, and downstream ingestion coordination. This decision reduces control-plane latency and improves operational resilience under network variability, while requiring explicit run tracking to observe completion.

Continuous acquisition is implemented as a constrained workflow rather than unconstrained crawling. The service resolves DOI candidates from PubMed responses and uses previous successful execution timestamps to support incremental acquisition semantics when publication metadata is available. This improves freshness-efficiency balance but remains sensitive to upstream metadata incompleteness.

Status derivation in scheduler execution is also explicit at both query and run levels. Query-level outcomes are computed from DOI-resolution and DOI-failure counts, while run-level outcomes are aggregated from query outcomes. This design improves interpretability of partial-success scenarios and avoids conflating upstream retrieval gaps with downstream ingestion errors.

3.1.5. Document lifecycle and processing pipeline

The lifecycle begins when a document is introduced through direct ingestion requests or scheduler-initiated fetch requests. At acquisition time, requests can be resolved by Digital Object Identifier (DOI) or PubMed Identifier (PMID); however, the indexing identity in the retrieval store is DOI-centered through normalized DOI to document identifier transformation.

Processing proceeds through asynchronous ingestion jobs. The architecture validates candidate items, reserves document identities to prevent conflicting writes, stores job and payload state in the current in-memory runtime, and dispatches workers for chunking, embedding, and vector indexing. Section-aware chunking and payload preservation ensure that downstream retrieval can reconstruct coherent evidence segments with provenance-bearing metadata, but the current job and payload stores are not restart-durable.

This pipeline prioritizes bounded request latency and operational recoverability over immediate index visibility. The tradeoff is eventual indexing completion, which requires explicit job polling for deterministic lifecycle observation.

Duplicate control is handled in two complementary layers: request-level idempotency keys and document-level reservation/commit semantics. This two-layer strategy mitigates duplicate writes arising from client retries and concurrent ingestion attempts. The tradeoff is additional state management overhead, including time-to-live policies for idempotency and payload stores.

3.1.6. Retrieval and answer generation pipeline

The retrieval pipeline transforms a user question into a ranked evidence context. Query embeddings are generated, vector candidates are retrieved under optional constraints, and candidate sets are reduced to bounded context windows before synthesis. This architecture constrains unbounded prompt growth and supports consistent runtime behavior under configurable limits.

The synthesis stage uses an LLM to generate structured responses with explicit citation linkage to retrieved context. An optional retrieval expansion mode, Hypothetical Document Embeddings (HyDE), generates a hypothetical passage from the question, embeds it, and merges those candidates with direct-question candidates before reranking [15]. The rationale is improved semantic recall for difficult or underspecified biomedical questions. The tradeoff is increased dependency on generation quality and higher response-time variance.

The ask flow further constrains synthesis reliability through explicit limits on candidate chunks, final chunk selection, and context length. These limits bound prompt size and stabilize runtime behavior under heterogeneous document lengths. The architectural implication is a controlled tradeoff between context breadth and deterministic resource usage.

3.1.7. State management and persistence

The platform stores different kinds of state according to their purpose and lifetime. Qdrant stores indexed chunk vectors and their metadata [44]. Each vector point is identified from the document identifier and chunk index, while the document identifier is derived from a normalized DOI. This DOI-centred identity links retrieval, document lookup, and deletion.

PostgreSQL stores scheduler queries, scheduler runs, per-DOI execution outcomes, and API audit records. These records must remain available after a process restart and support structured status queries. Relational storage provides this durability, but adds schema management and database availability as operational requirements.

Ingestion jobs, duplicate-control entries, request payloads, and queue items are stored in memory with bounded capacity and time-to-live settings. This keeps the asynchronous ingestion path simple, but these records are not durable across service restarts. Qdrant vectors that have already been written remain stored, while unfinished in-memory jobs may be lost.

YAML files store service and evaluation configuration, logs record runtime events, and the evaluation subsystem writes retrieval results, generated answers, metric reports, and configuration snapshots to run directories. Preserving these files allows later phases to use earlier outputs and supports analysis without repeating every model call. The tradeoff is that the files do not contain a complete Qdrant snapshot or the local Ollama model [45].

3.1.8. Operational observability

Observability is designed as a reliability and validity mechanism. Health and readiness endpoints separate process liveness from dependency readiness, enabling conservative admission behavior under partial infrastructure degradation. This distinction is particularly important when vector, database, or generation dependencies fail independently.

In parallel, request-context tracing and audit persistence provide reconstructable interaction histories, including request metadata, response status, and exception traces. The design improves post hoc diagnosis and experiment-time incident correlation. Its tradeoff is higher write overhead and stricter governance obligations for recorded payload content.

3.1.9. Architectural limitations

The architecture is constrained by external service behavior. Retrieval and synthesis quality depend on external model and infrastructure services beyond strict transactional control, while literature acquisition depends on upstream metadata fidelity. As a result, the system can provide explicit detection and recording of many failures, but cannot guarantee elimination of dependency-induced variance.

A second limitation is eventual consistency across acquisition and retrieval planes. Because acquisition and ingestion are asynchronous, recently identified documents may not be immediately available for retrieval. This is an intentional tradeoff that preserves bounded control-plane latency and operational isolation between serving and acquisition workflows.

Finally, stored workflow states and evaluation files support comparison between experiments, but they do not guarantee identical generated text. LLM-mediated synthesis can still vary under equivalent high-level inputs.

An additional limitation is sensitivity to identifier completeness in upstream biomedical sources. Scheduler-side DOI extraction and downstream document existence checks rely on consistent metadata availability; when identifiers are missing or inconsistently formatted, conservative failure or skip paths may reduce acquisition coverage for a given run window.

Taken together, the architecture favors explicit subsystem boundaries, inward dependency direction, and asynchronous workflow isolation to improve maintainability and reproducibility. The corresponding tradeoffs are eventual consistency, additional coordination overhead between services, and bounded but non-zero variance in LLM-mediated synthesis behavior.

3.2. Evaluation methodology

3.2.1. Overall evaluation methodology

The evaluation is designed to treat the platform as a set of separable capabilities: retrieval preparation and ranking, answer generation, answer support, risk factor extraction, robustness behavior, and operational runtime. This separation is necessary because a configuration can retrieve useful evidence but synthesize a weak answer, or it can produce a relevant-looking answer without evidence grounding. The evaluation design therefore reports each capability with its own metrics.

In this chapter, the following assumptions are made:

- *question set* refers to the natural-language questions,
- *query set* refers to those same inputs as curtailed during retrieval,
- *evaluation run* refers to one execution under a recorded configuration, and
- *benchmark* is reserved for the complete evaluation protocol rather than a question file or run directory.

The question set (Table 1) was developed to include questions related to the task of retrieving PubMed records that relate to risk factors and thrombosis, as well as questions are not related to such records.

Table 1. Question set used in the evaluation

Questions related to the task of retrieving PubMed records on thrombosis and risk factors (13/25)
1. How does upadacitinib compare to other active treatments regarding venous thromboembolism risk across immune mediated inflammatory diseases
2. Were there consistent safety signals indicating higher thromboembolic or cardiovascular risk with upadacitinib?
3. How do different deep vein thrombosis prophylaxis strategies affect wound healing and infection rates after total hip arthroplasty?
4. How did argatroban perform when combined with recombinant tissue plasminogen activator compared to thrombolysis alone?
5. Among catheter directed and systemic thrombolytic therapies, which intervention showed the most favorable mortality outcomes in submassive pulmonary embolism?
6. Which thrombolytic strategy was associated with the highest risk of major bleeding in submassive pulmonary embolism?
7. How did thrombolytic therapies impact pulmonary embolism recurrence and hospital length of stay?
8. How do short term oral anticoagulation and antiplatelet therapy compare in preventing device related thrombus after left atrial appendage closure?

9. Which anticoagulation strategy showed the most favorable balance between thrombotic prevention and bleeding risk after left atrial appendage closure?
10. Were there any safety signals indicating increased thromboembolic or cardiovascular risk associated with upadacitinib treatment?
11. How did MR black blood thrombus imaging compare with contrast enhanced MRI and noncontrast MR venography in diagnosing cerebral venous thrombosis?
12. Which thrombolytic strategies showed the best balance between mortality reduction and bleeding risk in submassive pulmonary embolism?
13. How reliable were existing risk prediction models for portal vein thrombosis after splenectomy, and what limitations were identified?

Questions not related to the task of retrieving PubMed records on thrombosis and risk factors (12/25)

1. What complications are significantly reduced when using integrated short peripheral catheters compared to non integrated catheters?
2. Which catheter related outcomes showed only non significant trends favoring integrated short peripheral catheters?
3. How did integrated catheter design influence the overall risk of catheter failure in hospitalized patients?
4. In which immune mediated inflammatory diseases was upadacitinib most frequently studied, and what were the overall safety conclusions?
5. What evidence exists comparing rivaroxaban and enoxaparin regarding postoperative wound complications?
6. Does intraoperative drain use influence wound healing outcomes in total hip arthroplasty?
7. What evidence supports or refutes the efficacy of argatroban in improving functional outcomes after acute ischemic stroke?
8. Does argatroban increase the risk of intracranial hemorrhage or major bleeding compared to standard therapy?
9. How does left ventricular unloading with Impella during extracorporeal cardiopulmonary resuscitation affect survival and neurologic outcomes?
10. In which patient populations was combined ECMELLA support more frequently used compared to venoarterial ECMO alone?
11. What tradeoffs were observed between mortality benefit and complication rates with ECMELLA support during extracorporeal cardiopulmonary resuscitation?
12. Which catheter related complications were significantly reduced by integrated short peripheral catheters compared with non integrated catheters?

The evaluation compares three answer-generation configurations.

- The LLM-only baseline sends each query directly to the generation model without retrieved contexts. It measures parametric answer behavior and provides a comparison point for answer relevancy, but context-dependent metrics such as faithfulness and context precision are not applicable because no retrieved contexts are supplied.
- RAG without HyDE routes each question through the biomedical RAG API and synthesizes an answer from retrieved evidence without generated query expansion.

- RAG with HyDE enables an additional expansion step in which a hypothetical answer document is generated, embedded, and used to retrieve additional candidates before merging and reranking.

Retrieval evaluation asks whether the system ranks relevant biomedical evidence near the top of the result list. This is different from asking whether the final answer is well written. A retrieval system can find relevant papers while the generator still writes a weak answer, and a generator can write a plausible answer even when retrieval was poor. The independent retrieval metrics therefore evaluate the ranked evidence list before answer synthesis. These metrics are computed during the evaluation phases Candidate Pooling (Phase 1) and Retrieval Evaluation (Phase 2), which evaluate ranked document lists against manually annotated relevance judgments (Qrels).

3.2.2. Thrombosis literature dataset

The evaluation use case is based on a literature dataset prepared within the ThrombUS+ project with the aim to identify risk factors related to thrombosis. The dataset includes approximately 8,500 records retrieved from PubMed on 17 May 2025 via a PubMed query designed by ThrombUS+ medical experts to retrieve systematic reviews and meta-analyses related to deep vein thrombosis and thromboembolism. These records were then reviewed by the medical experts for relevance to venous thrombosis and risk factors. Criteria for a record to be relevant included: the record reported on venous thrombosis and the record reported on risk factors.

The annotation work was performed by the ThrombUS+ medical team (as acknowledged in the Contributions section) using the Label Studio [46] appropriately configured for this task. Annotations included title and abstract phrases that indicate to the annotator that the abstract relates to venous thrombosis, or relates to risk factor or relates to particular clinical topics, and confidence self-ratings. These fields provide structured review judgements that can be examined alongside each PubMed record.

For this thesis the literature dataset subset used included 321 assessed records: 182 non-relevant, 108 relevant, and 31 highly relevant. Annotations where exported from the Label Studio API as JSON task objects, and these were further enriched by the processing software with bibliographic identifiers and converted to the biomedical platform's ingestion schema.

3.2.3. Retrieval evaluation workflow

The retrieval evaluation contains 25 queries, 250 direct-retrieval rows, 250 HyDE-retrieval rows, 300 final-context rows, and 321 pooled candidates. It also contains a completed relevance-assessment file, per-query retrieval metrics, and aggregate comparison tables. Retrieval ranking results are derived from these recorded evaluation outputs.

The candidate pool combines Direct Retrieval and HyDE Retrieval candidates before relevance assessment. Pooling candidates from both modes into one set supports a fair comparison because both ranked lists are scored against the same relevance judgements.

Figure 2 summarizes the retrieval procedure. It separates retrieval from answer generation, pools Direct Retrieval and HyDE Retrieval candidates before relevance assessment, hides retrieval metadata during review to reduce ranking bias, validates the completed judgements, writes qrels.tsv, and computes mean reciprocal rank (MRR), Precision@k, Recall@k, and nDCG@k (Normalized Discounted Cumulative Gain) for both retrieval modes against the same reference file.

The extraction evaluation uses a separate derived reference set. This set ties scoring to reviewed publication records, citation alignment, confidence policy, and canonical factor matching. It is not an adjudicated clinical gold standard. Extraction metrics therefore describe behavior under the documented evaluation protocol, not clinical correctness.

The evaluation uses a phase-separated process that writes intermediate files for later analysis. This separates retrieval ranking from answer quality and factor extraction. Table 2 summarizes the six evaluation phases. Each metric can be traced to the phase and output file that produced it.

Table 2. Overview of the evaluation phases

Phase	Name	Purpose	Input / Output	Metrics
Phase 1	Candidate Pooling	Extract candidate documents from Direct and HyDE retrieval to build a pooled set for relevance assessment.	**I:** Queries, RAG API **O:** pooled_candidates.jsonl	N/A
Phase 2	Retrieval Evaluation	Compute Information Retrieval (IR) metrics for ranked document lists against the recorded relevance judgements.	**I:** retrieval_*.jsonl, qrels.tsv **O:** retrieval_metrics.json	MRR, nDCG@k, P@k, R@k
Phase 3	Answer Generation	Execute the RAG pipeline or baseline LLM to generate answers for the query set.	**I:** Queries, RAG API, Ollama API **O:** phase3_answers_*.jsonl	Mean Latency
Phase 4	RAGAS Evaluation	Assess answer quality using automated multi-dimensional metrics for RAG systems.	**I:** phase3_answers_*.jsonl, Ollama API **O:** phase4_ragas_*.jsonl	Faithfulness, Relevancy
Phase 5	Consistency Audit	Sample generated outputs to support manual audit and verify internal system consistency.	**I:** phase3_answers_*.jsonl **O:** audit_sample.json	N/A
Phase 6	Extraction Evaluation	Compare extracted risk factors against a reference set derived from reviewed annotations.	**I:** phase3_answers_*.jsonl, tasks_clean.json **O:** phase6_extraction_*.csv	Precision, Recall, F1

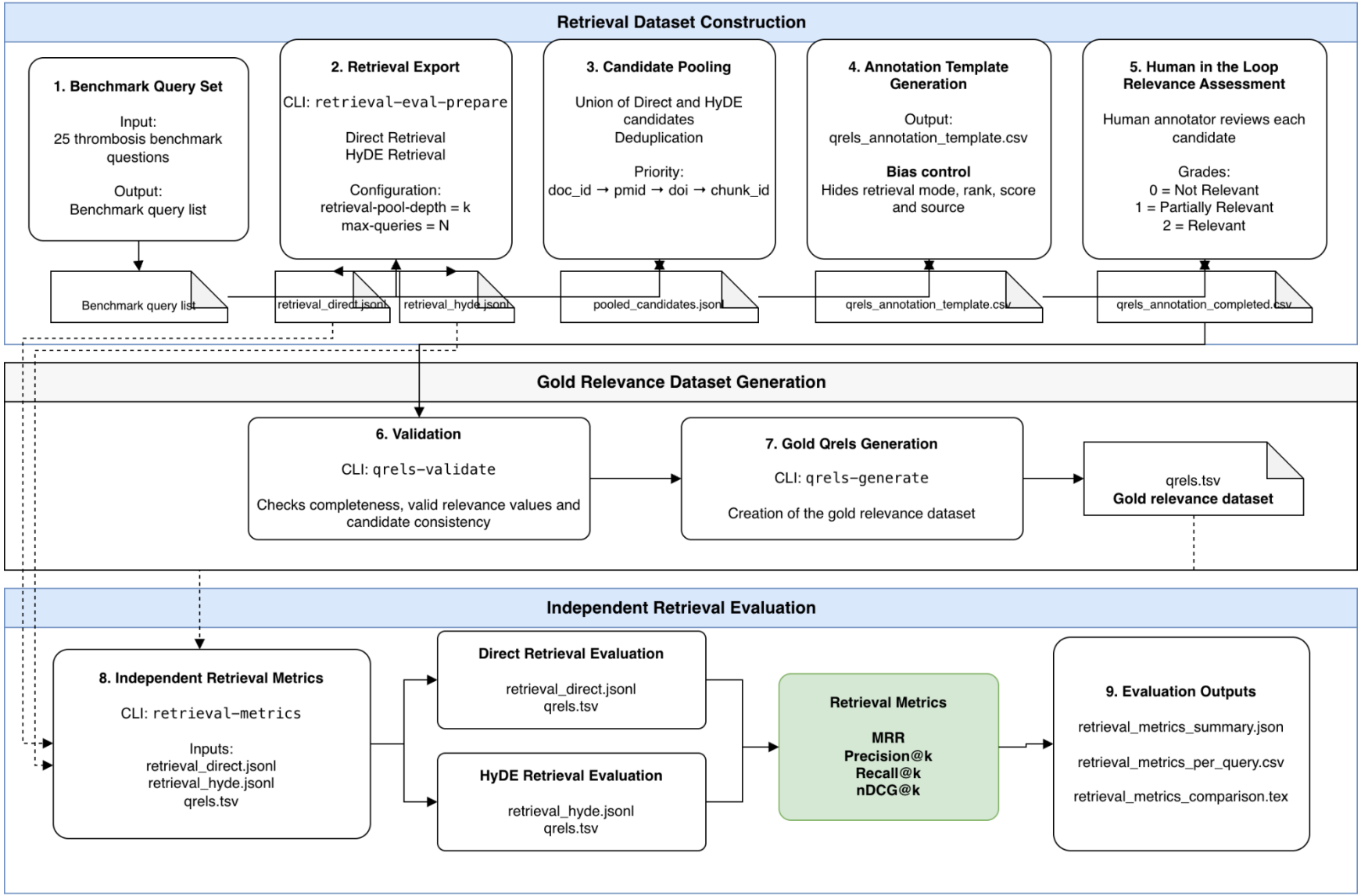

Figure 2. Retrieval evaluation workflow used to pool candidates, record relevance judgements, and compute Direct Retrieval and HyDE Retrieval metrics.

Each phase writes its output before the next phase runs, so generated answers, RAGAS scores, extraction summaries, and evaluation summaries can be examined independently.

3.2.4. Experimental configuration

The primary evaluation uses 25 thrombosis-oriented questions and three answer-generation configurations: LLM only, RAG without HyDE, and RAG with HyDE. The recorded experimental settings use seed 42 and temperature 0.0. A fixed seed was selected to control stochastic variation, and 42 is simply the conventional value chosen for this evaluation; it has no scientific significance. Temperature 0.0 reduces sampling variability so that observed differences between configurations are less affected by random token selection.

Extraction evaluation depends on the availability of derived reference factors from the reviewed annotations. Under the configured no-gold policy, questions without derived reference factors are skipped rather than counted as failures. Extraction results therefore describe only the covered subset of the 25 questions.

The robustness evaluation uses seeds 11, 22, 33, 44, and 55 at temperature 0.2. These are distinct fixed values selected to observe variation across repeated runs while keeping evaluation cost manageable; no mathematical significance is assigned to them. The small nonzero temperature introduces controlled stochastic variation without making generation highly random.

The implementation does not propagate these controls equivalently through all configurations. Phase 3 passes temperature and seed to the direct Ollama call used by the LLM-only baseline. The RAG API calls do not receive equivalent request-level controls; synthesis uses service-controlled deterministic options, while HyDE uses deterministic options and caching. The robustness results can therefore measure stochastic variation for LLM only, but zero variation in the RAG paths cannot be interpreted as equivalent stochastic robustness.

3.2.5. Metrics

The evaluation uses several metric families because the platform performs several different functions. A retrieval metric asks whether useful evidence appears near the top of a ranked list. A generation metric asks whether an answer is relevant to the question or supported by supplied contexts. An extraction metric asks whether the system recovered a set of risk factors. Runtime records the cost of the evaluation. These measures are related, but they are not interchangeable. A high retrieval score does not imply a faithful answer, and a relevant answer does not imply correct risk-factor extraction.

All reported scalar metrics are interpreted with their denominator in mind. A score of 0.80 does not mean “80 percent clinically correct”. Precision@5 counts only the top five retrieved items, micro F1 counts individual factor decisions across covered queries, and answer relevancy is an automated score over generated answers. Each value is meaningful only when paired with the measured output, reference information, and experimental configuration.

The simplest way to understand a metric is to ask what mistake it punishes. Precision punishes extra unsupported items. Recall punishes missed relevant items. F1 punishes imbalance between precision and recall. Faithfulness punishes answer statements that are not supported by the supplied contexts. Context precision without reference punishes contexts that appear less useful for supporting the generated answer. Latency punishes slow execution. The metrics are reported separately so that the reader can see which kind of mistake each configuration tends to make.

Each metric has an explicit evidence boundary. The RAGAS, extraction, robustness, and runtime values belong to the June 2026 generation evaluation. The retrieval-ranking values belong to the separate retrieval evaluation. Both use the same 25 questions, but they answer different evaluation questions and should not be collapsed into one aggregate system score.

The most important boundary is between automated scoring and biomedical truth. RAGAS scores are automated evaluator outputs. Extraction scores compare against a derived reference set. Retrieval metrics depend on qrels and reflect one completed relevance file. None of these files proves clinical correctness or replaces adjudicated biomedical review.

3.2.5.1. Retrieval metrics

Precision at k measures how many of the top k returned items are relevant. If the system returns five papers and three are relevant, Precision@5 is 3/5, or 0.60.

Recall at k measures how many of the known relevant items were recovered within the top k. If a query has four relevant papers in the reference set and the system retrieves two of them in the top five, Recall@5 is 2/4, or 0.50.

Mean Reciprocal Rank (MRR) measures how early the first relevant result appears. The reciprocal rank for a query is 1.0 when the first result is relevant, 0.5 when the first relevant result is second, and approximately 0.33 when it is third. MRR averages this value across queries. It is useful when the first useful result matters strongly, such as when a reviewer wants to see at least one relevant source immediately.

Normalized Discounted Cumulative Gain at k (nDCG@k) rewards rankings that place more useful documents earlier [26]. Unlike binary precision and recall, nDCG can incorporate graded relevance, so a highly relevant document at rank 1 is worth more than a weakly relevant document at rank 5. Intuitively, nDCG asks whether the ranking order is good, not just whether relevant items appear somewhere in the cutoff.

These four retrieval metrics should be read together. Precision@5 emphasizes top-five purity, Recall@5 emphasizes top-five coverage, MRR emphasizes the first useful hit, and nDCG@5 emphasizes graded ordering. No single retrieval metric captures all aspects of evidence ranking.

3.2.5.2. Generation and RAGAS metrics

Generation quality is evaluated using Retrieval Augmented Generation Assessment (RAGAS) metrics [29]. Answer relevancy estimates whether the response addresses the user's question. Faithfulness estimates whether the generated response is supported by supplied contexts. Context precision without reference estimates whether the retrieved contexts appear useful for supporting the produced answer without using a separate gold answer. It is an answer-support proxy, not an independent retrieval ranking metric and not proof of biomedical correctness. Answer Generation (Phase 3) and RAGAS Evaluation (Phase 4) provide the basis for evaluating answer quality and support.

Answer relevancy can be understood as a question-answer alignment score. If the question asks about thrombosis risk factors and the answer discusses those risk factors directly, answer relevancy should be higher. If the answer drifts into unrelated background material, answer relevancy should be lower. This metric is useful for detecting whether the response addresses the prompt, but it does not require the answer to cite or obey retrieved evidence. For that reason, the LLM-only baseline can receive an answer relevancy score even though it has no retrieved contexts.

Faithfulness measures a different property. It asks whether statements in a generated answer are supported by the contexts supplied to the generator. In simple terms, if the context says “factor A is associated with thrombosis” and the answer says “factor A is associated with thrombosis”, that statement is supportable. If the answer adds a factor or causal claim that is not present in the supplied context, faithfulness should decrease. Faithfulness is therefore central to evidence-grounded RAG evaluation, but it is not applicable to the LLM-only baseline because no retrieved contexts are supplied.

Context precision without reference is more subtle. It estimates whether the contexts supplied to the answer appear useful for supporting the answer, without comparing the contexts to an independent reference answer. A high value can indicate that the answer is focused around the retrieved material. It is not the same as Precision@5 from retrieval evaluation. Precision@5 uses relevance judgements over ranked documents, while context precision without reference is an automated answer-support proxy computed during RAGAS evaluation.

3.2.5.3. Risk factor extraction metrics

Risk factor extraction is evaluated separately from general answer quality. Extraction Evaluation (Phase 6) compares predicted risk-factor sets with reference factors derived from reviewed annotations and cited documents. True positives count recovered reference factors, false positives count predicted factors absent from the derived reference set, and false negatives count reference factors missed by the system. Precision captures over-extraction, recall captures missed factors, and F1 balances the two.

A true positive is a correctly recovered risk factor. If the derived reference set contains “prior venous thromboembolism” and the system extracts the same canonical factor, that is a true positive. A false positive is an extra factor predicted by the system but not present in the derived reference set. A false negative is a derived reference factor that the system failed to extract. These counts are the raw building blocks of precision, recall, and F1.

Precision asks, “When the system extracts a factor, how often is that extraction in the reference set?” If a system extracts 10 factors and 7 are correct, precision is 7/10, or 0.70. Low precision means the system creates too many unsupported extractions, which increases human review burden. Recall asks, “Of the factors that should have been extracted, how many did the system find?” If the reference set contains 8 factors and the system finds 6, recall is 6/8, or 0.75. Low recall means the system misses known factors. F1 combines precision and recall so that a system is penalized when one is high and the other is low.

Micro and macro averaging answer different aggregation questions. Micro precision, recall, and F1 pool all individual factor decisions across the covered queries before computing the metric. This means queries with more factors can have more influence. Macro precision, recall, and F1 compute per-query scores first and then average them, giving each covered query more equal influence. Reporting both views makes the extraction result less dependent on a single aggregation convention.

Chapter 4| Results

4.1. System implementation

In implementation, request lifecycle management is explicitly layered. Middleware components attach request identifiers, perform access logging, and capture audit events before and after endpoint execution, while use cases handle domain-specific validation and workflow orchestration. This partition makes operational accountability concrete in the running service by separating cross-cutting concerns from business-flow logic.

The scheduler service applies the same implementation pattern through its own API, domain, use-case, adapter, and repository layers. In practice, this symmetry gives both services a consistent structure for cross-service workflows and aligns their error and status semantics across ingestion-oriented and acquisition-oriented operations.

Overall system implementation was based on Python FastAPI, PostgreSQL, and qDrant DB. The code is available at: https://github.com/MSc-Biomedical-Informatics-DUTH-ATHENA/2026_ChaikalisN_PubMed-RAG

Technical details on implementation and configuration are available in Annex 1-7.

4.1.1. Literature records annotation service

The labelstudio-tools package implements the software connection between abstract preparation, Label Studio [46], exported annotations, and RAG ingestion. It converts reviewable PubMed records into Label Studio task objects, retrieves reviewed task objects, adds missing bibliographic identifiers, and writes a clean set of publication records for the biomedical platform API. This processing is offline and is not part of query-time retrieval.

Configuration is environment-driven. The package resolves a project root from the active .env file, configures logging centrally, and exposes variables for Label Studio connectivity, Excel input location, filter toggles, export destinations, and RAG ingestion behavior. This design keeps the implementation portable across local runs and batch executions, while making behavior explicit enough to reproduce the same data-processing path under controlled settings.

The task-preparation entry point loads a structured Excel worksheet into a data frame. The worksheet is read from the configured EXCEL_FILE path and the sheet named After Automatic Exclusion. Each row represents a candidate PubMed record for thrombosis risk-factor review. The required columns are PMID, ArticleTitle, and Abstract; these provide the PubMed identifier, article title, and abstract text used to create the Label Studio task. The worksheet may also contain screening and classification columns, including Excluded, Not related to venous thrombosis, Reporting on risk factors, Narrative Review, Systematic Review, Meta Analysis, and category columns such as Category 1 through Category 5. The Boolean-like screening columns are normalized into consistent truth values before filtering. Depending on configuration, the filtering stage can exclude pre-marked records, records declared unrelated to venous thrombosis, or records not reporting on risk factors. After optional sampling, each valid row is transformed into a minimal Label Studio task containing PMID, title, and abstract content; rows without PMID are skipped to avoid generating invalid review items.

Upload is implemented through the Label Studio Software Development Kit (SDK). Before importing, the uploader checks authentication and verifies that the configured project identifier is visible to the current

account. Label Studio task objects are then sent in fixed-size batches, with exponential retry backoff when a batch import fails. A temporary network or service failure therefore does not require the complete import to restart.

The export process uses the Label Studio API to page through project task objects, refresh an access token when needed, and collect the reviewed records. Exported task objects are marked as labelled, but the software exports and cleans annotations; it does not adjudicate them. DOI enrichment is followed by validation of the fields required for ingestion: source, source identifier, DOI, year, journal, authors, title, and abstract. The implementation writes separate raw, clean, rejected, and missing-identifier JSON files atomically. Data-quality failures can therefore be examined without repeating the complete export.

When clean publication records are available, an integration script converts each one into the ingestion schema expected by the biomedical platform. Only records with complete DOI, year, title, journal, author list, and abstract fields are converted. They are sent in batches to the /v1/ingest/items endpoint, with retry handling for transport failures, rate-limit responses, and server-side failures. A short delay after successful requests reduces immediate rate-limit pressure on consecutive batches.

4.1.2. Ingestion and embedding services

The ingestion strategy is implemented in the biomedical RAG service. At the core of the indexing pipeline is a domain-specific SectionAwareChunker. Unlike naive token-based chunkers, this component respects biomedical document structure. It actively detects and filters out non-informative sections, such as bibliographies, ensuring that only evidence-bearing text is embedded and indexed. Embeddings are generated locally using the SentenceTransformersEmbeddingProvider, which transforms the chunks into dense vectors before they are stored in the Qdrant vector database [20],[44].

The retrieval logic, orchestrated by the AskUseCase, uses a multi-stage hybrid system rather than relying exclusively on dense vector search. The pipeline begins with an optional retrieval expansion phase using Hypothetical Document Embeddings (HyDE), which generates a hypothetical passage and retrieves additional candidates from the same vector index [15].

Following candidate generation, the system uses a hybrid ranker to fuse signals. This ranker combines dense vector ranks, optional HyDE ranks, and in-memory lexical BM25-style ranks through reciprocal-rank fusion [47]. It also applies section weights; for example, evidence located in results or conclusions sections receives higher weight than methods or supplementary sections. This domain-oriented ranking policy biases the final context window toward more evidence-bearing sections, but it does not guarantee that clinically most important evidence is always selected.

Once the top-ranked contexts are finalized, the system enforces a strict JSON-only prompt during synthesis. The application layer performs post-generation parsing, schema validation, and citation backfilling so that extracted risk factors are linked to provided context chunks when possible. This validation layer reduces unsupported output and improves compatibility with downstream evaluators, but it does not eliminate hallucination risk or establish clinical correctness.

4.1.3. PubMed scheduler

The scheduler implements continuous acquisition as an asynchronous run rather than a blocking request. A configured UTC schedule or manual API call first stores a run record and returns. A background process then

searches PubMed for each enabled query, filters results against the last successful run time, resolves unique DOIs, checks each DOI against the biomedical RAG service, submits unseen documents through the batch-fetch endpoint, and polls the ingestion job until completion. Run-level, query-level, and per-DOI outcomes are stored separately, allowing partial success to be distinguished from complete failure.

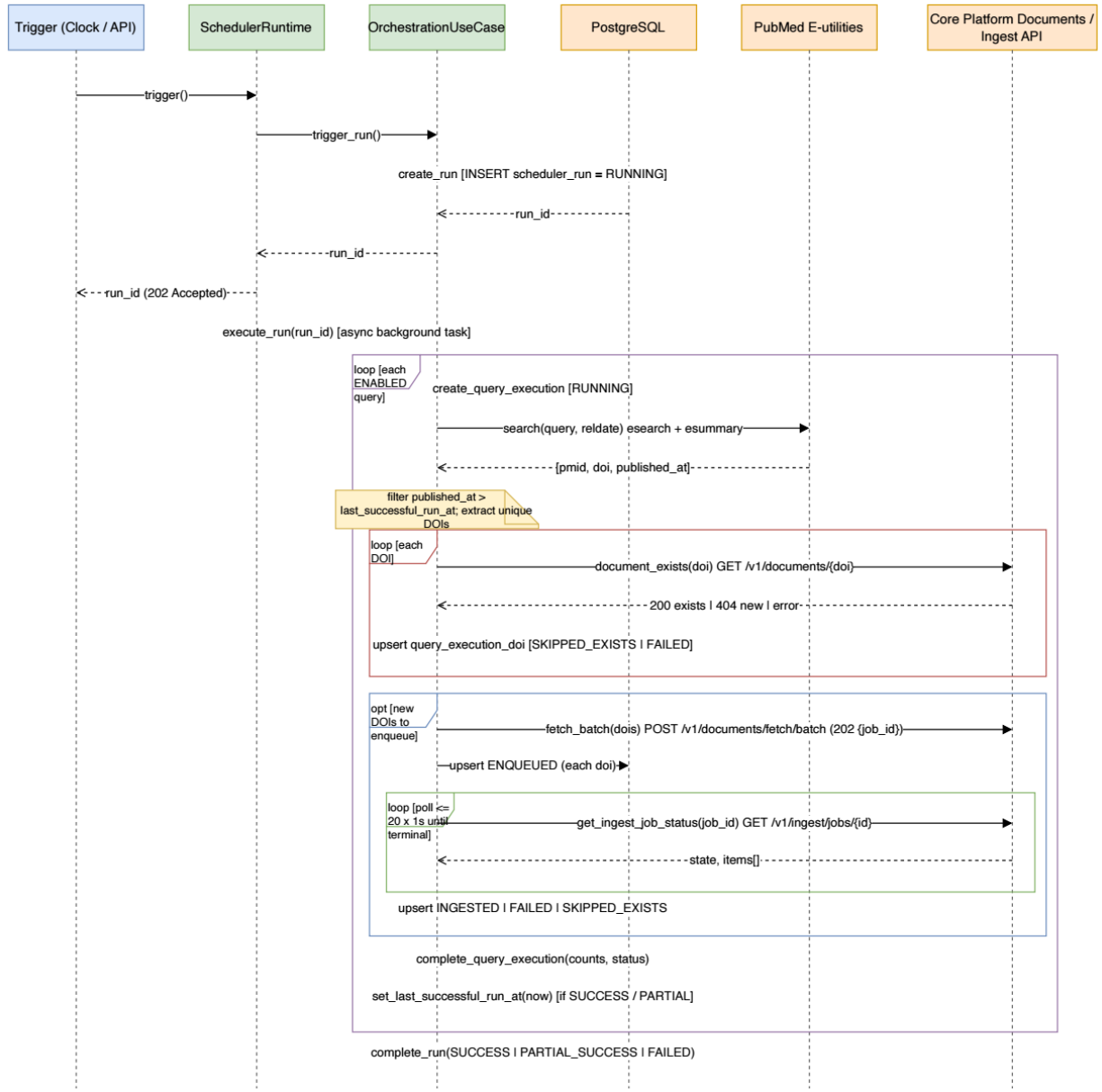

Figure 3. Sequence of a PubMed scheduler run: trigger and asynchronous hand-off, per-query PubMed search with incremental filtering, per-DOI existence checks, batch ingestion, ingest-job polling, and stored run outcomes.

Figure 3 summarizes the sequence of a PubMed scheduler run: trigger and asynchronous hand-off, per-query PubMed search with incremental filtering, per-DOI existence checks, batch ingestion, ingest-job polling, and stored run outcomes. Two implementation properties are visible in it. First, the acquisition control plane is decoupled from the ingestion data plane: the scheduler never writes to the vector store directly, but coordinates document existence checks, batch fetches, and ingest-job polling through the biomedical RAG

API. Second, the incremental watermark is only advanced when a query execution completes successfully or partially, which prevents a failed run from silently skipping publications in the next acquisition window.

4.2. System Configuration

The system uses external configuration for retrieval depth, embedding identity, vector-store endpoints, local model endpoints, scheduler integration, and evaluation settings. These values affect experimental interpretation. The implementation therefore distinguishes settings that are actively used, settings reserved for interface compatibility, and settings read only by the evaluation process.

Externalized configuration improves reproducibility by making assumptions visible, but it does not by itself guarantee deterministic behavior. Exact rerun identity can still depend on vector-store state, local model files, evaluator runtime, corpus contents, and service versions. Configuration therefore records settings for controlled comparison; it does not guarantee perfect future replay.

4.2.1. RAG service configuration

The rag.yaml file records the core retrieval and indexing assumptions of the biomedical RAG service. It configures the sentence-transformer embedding provider, CPU execution, embedding normalization behavior, embedding batch size, chunk size, chunk overlap, vector backend, and nominal retrieval depth. These values matter because they determine the embedding space and chunk granularity from which Qdrant candidates are produced [44]. For a non-technical reader, this file defines how documents are broken into retrievable pieces and how those pieces are represented before search begins.

This file does not configure HyDE activation or generation temperature. HyDE behavior is controlled through the LLM configuration and the X-HyDE-Enabled request header used by the evaluation client. The evaluation configuration records deterministic and robustness seeds and temperatures, but the execution path applies those controls differently across modes. The LLM-only baseline receives request-level seed and temperature directly from the evaluation CLI, whereas the current RAG API ask path does not expose equivalent request-level seed or temperature controls for RAG synthesis or HyDE generation.

4.2.2. LLM and HyDE configuration

The llm.yaml file configures the local Ollama dependency used by the biomedical RAG service [45]. It records the Ollama base URL (Uniform Resource Locator), the generator model identifier, the HyDE model identifier, the default HyDE switch, question and context bounds, candidate and final chunk limits, concurrency, retry count, and timeout. HyDE is disabled by default and can be enabled per request through the API header used in the evaluation configurations.

The LLM configuration also contains reranker-related fields. The implemented ranking path combines dense vector rank, optional HyDE rank, lexical BM25-style rank, and section weighting. The reported evaluation does not use an LLM reranker. A configuration key alone is therefore not evidence that the associated behavior was active in the experiment.

4.2.3. Vector store configuration

The qdrant.yaml file defines the Qdrant REST endpoint, optional gRPC endpoint, timeout, collection naming prefix, distance metric, and telemetry flag [44]. Collection names are derived from the configured prefix and

embedding model identifier, which supports model-specific collections without changing the external API contract.

The configuration sets cosine distance and disables gRPC preference. This matters for reproducibility because vector-store behavior is a function of both embedding representation and collection configuration. Changing the embedding model, normalization setting, or collection naming policy can create a different retrieval corpus even when the same API endpoint is used.

4.2.4. Scheduler configuration

The PubMed scheduler configuration is stored in pubmed_scheduler.yaml. It enables the scheduler, records the UTC execution times, and defines the HTTP integration paths used to communicate with the biomedical RAG service. The scheduler therefore remains a separate acquisition subsystem: it owns acquisition timing and RAG API coordination, but it does not directly mutate the vector store.

The scheduler configuration is not a generic cron file. It contains explicit UTC schedule times and API paths for document lookup, batch fetch, and ingest-job polling. Query definitions and run outcomes are stored through the scheduler domain and persistence layer rather than in a single flat configuration file.

4.2.5. Evaluation configuration

The eval.yaml file defines the experimental control surface for the evaluation subsystem. It records the RAG API endpoint, ask-mode HyDE header, Ollama model settings, deterministic and robustness seeds, query file, RAGAS embedding model, RAGAS embedding execution device, extraction policy, and metric k values. These settings define the comparison between LLM-only, RAG without HyDE, and RAG with HyDE configurations.

The evaluation CLI writes configuration snapshots into run directories. These snapshots link generated answers, RAGAS outputs, extraction summaries, and aggregate results to their execution settings. They do not archive the complete vector database or Ollama runtime.

4.2.6. CUDA evaluation execution support

The evaluation pipeline also supports an optional CUDA (Compute Unified Device Architecture, VIDIA's parallel computing platform and programming model) execution profile for the evaluation-side embedding workload used during RAGAS scoring. This is an execution-environment option for the evaluation subsystem rather than a change to the reported metric definitions or to the scientific interpretation of those metrics, and it does not imply that every subsystem executed on the same hardware path. The deployment mechanics and environment-validation procedure are given in Appendix A.

4.2.7. Configuration boundaries and reproducibility

The RAG service exposes request-level search depth, and the search use case uses a value from the request model when none is supplied. The rag.yaml retrieval top_k value is therefore a nominal service setting rather than proof that every search used that value. Similarly, the response schema includes a cursor field, but the current search use case returns no next cursor.

Configuration files can contain reserved or partially connected settings. Reported results must therefore be tied to recorded evaluation files and the implemented execution path, not only to the presence of a YAML

key. The recorded settings do not establish equivalent stochastic control between the LLM-only and RAG paths.

The recorded evaluation stores a configuration snapshot alongside generated answers and metric files. This links a reported metric to the settings that produced it. Local Qdrant state, local Ollama model files, service uptime, PubMed availability, and selection of evaluation outputs can still affect future runs. Stronger reproducibility would require corpus version identifiers, vector-store snapshots, model and runtime digests, request-level stochastic controls for RAG and HyDE, and expanded retrieval results with adjudicated qrels.

4.2.8. Error handling

Error handling in annotation-data processing uses fail-fast validation and bounded retries. Invalid Excel inputs, missing required environment variables, missing project access, empty publication sets, and malformed Label Studio task objects raise explicit errors. Operations exposed to network variability use retry loops with exponential backoff, including Label Studio batch import and RAG ingestion requests. The implementation therefore distinguishes non-recoverable configuration or data-format errors from temporary transport or service failures.

4.3. System execution environment

The platform uses a local-first, Docker Compose deployment [48]. The biomedical RAG service runs alongside Qdrant and PostgreSQL. Qdrant stores document-chunk vectors and metadata, while PostgreSQL stores audit records. The FastAPI service exposes ingestion, retrieval, synthesis, and document-management endpoints.

The language model is served separately through Ollama [45]. The biomedical RAG service calls the model through HTTP, which keeps model hosting outside the application process. The PubMed scheduler runs as an independent service and communicates with the biomedical RAG service through its document and ingestion endpoints.

The evaluation subsystem runs as a command-line process. It reads an evaluation profile, calls the RAG and Ollama endpoints, and writes intermediate files and configuration snapshots to a run directory. An optional CUDA profile can move the evaluation-side embedding workload used by RAGAS scoring from CPU (Central Processing Unit) to CUDA; this does not imply that every subsystem or reported evaluation run used the same hardware path.

This execution model supports local analysis and repeatable configuration, but it does not provide production orchestration, autoscaling, or high availability. Exact reruns can also depend on the Qdrant collection, local model files, service availability, and PubMed responses. Detailed Compose definitions, ports, commands, CUDA checks, and run identifiers are provided in Appendix A.

4.4. Evaluation results

The results describe one biomedical RAG implementation and experimental configuration, not all retrieval-augmented biomedical systems. The primary evaluation does not identify one universally superior configuration. The strongest result depends on the measured dimension.

Metric disagreement is expected. The primary run gives the LLM-only baseline the highest answer relevancy, RAG with HyDE the highest context precision without reference, and RAG without HyDE the highest

faithfulness and extraction scores. These outcomes are not contradictory because each metric asks a different question. The results are therefore reported as tradeoffs rather than as one “best model”.

This distinction matters because retrieval, generation, grounding, extraction, runtime, and robustness describe different behaviours. The results are used to diagnose these stages separately rather than to produce one aggregate system score.

4.4.1. Retrieval

Table 3 compares direct and HyDE retrieval compared against the same qrels file. HyDE increased MRR from 0.865 to 0.910 and Recall@5 from 0.5747 to 0.6174, so the first relevant result tended to appear earlier and more of the judged relevant set appeared within the first five results. Direct retrieval had higher Precision@5, 0.552 versus 0.544, and higher nDCG@5, 0.7590 versus 0.7231. Its top-five list contained a slightly larger relevant share and followed the graded relevance order more closely.

Table 3. Retrieval ranking metrics for the 25-query evaluation.

Metric	Direct Retrieval	HyDE Retrieval
MRR	0.865	0.910
Precision@5	0.552	0.544
Recall@5	0.5747	0.6174
nDCG@5	0.7590	0.7231

The recorded results favor HyDE for early-hit discovery and top-five coverage, while direct retrieval has higher top five precision and graded ordering. They do not establish one retrieval mode as better for every query or use.

One possible explanation is that HyDE’s generated passage bridges terminology differences for some questions, while also introducing terms that change the graded order. The evaluation did not test query types separately, so the results do not justify assigning HyDE to broad searches or direct retrieval to high-quality evidence ranking without further analysis.

The retrieval values reflect the combined effect of Qdrant vector search, the selected embedding model, chunk construction, rank fusion, and section weighting. They apply to the recorded candidate pool and relevance judgements. They do not isolate the contribution of any one component.

4.4.2. Generation and RAGAS

Table 4 and Figure 4 show RAGAS metrics by answer-generation configuration. LLM only has the highest answer relevancy, 0.564609, but this does not establish grounding because no retrieved contexts are provided. RAG with HyDE has slightly higher answer relevancy than RAG without HyDE, 0.539485 versus 0.529602, and higher context precision without reference, 0.826667 versus 0.686667. RAG without HyDE has higher faithfulness, 0.685810 versus 0.480571. These values do not show that RAG always improves answer relevancy or that one RAG configuration leads on every RAGAS metric.

Table 4. Primary RAGAS metrics by answer-generation configuration.

Configuration	Answer relevancy	Faithfulness	Context precision without reference
LLM only	0.564609	N/A	N/A
RAG without HyDE	0.529602	0.685810	0.686667
RAG with HyDE	0.539485	0.480571	0.826667

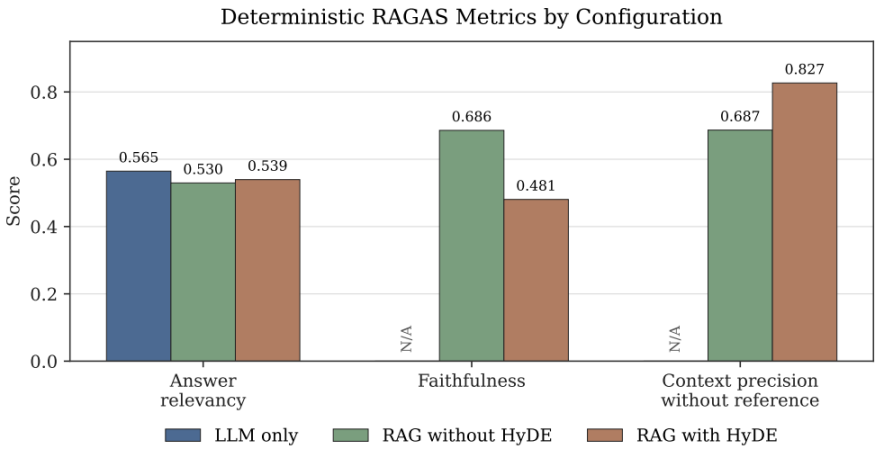

Figure 4. Primary RAGAS metrics by answer-generation configuration.

The LLM-only score exceeds the RAG-with-HyDE score by 0.025124 and the RAG-without-HyDE score by 0.035007. Retrieved context did not increase automated answer-question alignment in this run. Faithfulness and context precision are not applicable to LLM only because those measures require retrieved contexts. The result therefore concerns question-answer alignment only and does not make LLM only the strongest biomedical system.

The faithfulness comparison points in the opposite direction. RAG without HyDE scores 0.685810, while RAG with HyDE scores 0.480571, a difference of 0.205239. Since faithfulness measures support from supplied contexts, the recorded result favors RAG without HyDE for this dimension.

Context precision without reference favors HyDE: 0.826667 versus 0.686667, a difference of 0.140000. The automated evaluator judged the HyDE contexts to be more useful for the produced answer. This is an answer-support measure, not qrels-based retrieval precision

The LLM-only result is a baseline rather than a preferred biomedical architecture. It shows that the local model produced answers judged relevant to the questions. It does not show that those answers are traceable to a controlled corpus, supported by cited evidence, or suitable for clinical use.

HyDE has higher context precision but lower faithfulness than direct RAG. One possible explanation is that the hypothetical passage changes the retrieved context in a way that is useful for the generated answer but does not support every answer statement. The experiment did not isolate hypothetical-passage quality, candidate fusion, final context selection, and synthesis, so it cannot identify one causal mechanism. For the recorded faithfulness measure, RAG without HyDE performs better.

4.4.3. Risk factor related document extraction

Table 5 shows higher values for RAG without HyDE on all reported extraction measures: micro precision, micro recall, micro F1, macro precision, macro recall, and macro F1. RAG without HyDE recovers 38 true positives rather than 34. Both configurations produce 23 false positives and 3 false negatives, so the evaluation does not show a reduction in false positives from HyDE.

Table 5. Risk-factor related document extraction evaluation metrics for the 13 covered questions.

Configuration	RAG without HyDE	RAG with HyDE
Evaluation queries	13	13
Coverage	0.52	0.52
TP	38	34
FP	23	23
FN	3	3
Micro P	0.6229951	0.596491
Micro R	0.926829	0.918919
Micro F1	0.745098	0.723404
Marco P	0.637912	0.601282
Macro R	0.939560	0.923077
Macro F1	0.731241	0.700821

The counts explain the metric difference. Both RAG configurations made the same number of false-positive and false-negative mistakes: 23 false positives and 3 false negatives. The decisive difference is true-positive recovery. RAG without HyDE recovered 38 true positives, while RAG with HyDE recovered 34. Because the false-positive burden is equal, the additional four true positives improve direct RAG's precision, recall, and F1.

The micro precision values can be reproduced directly from the counts. For RAG without HyDE, precision is 38 divided by 38 plus 23, which is 38/61, or 0.622951. For RAG with HyDE, precision is 34 divided by 34 plus 23, which is 34/57, or 0.596491. In plain terms, both systems produced extra factors, but direct RAG produced more correct factors for the same false-positive count.

The micro recall values can also be reproduced from the counts. For RAG without HyDE, recall is 38 divided by 38 plus 3, which is 38/41, or 0.926829. For RAG with HyDE, recall is 34 divided by 34 plus 3, which is 34/37, or 0.918919. Both systems missed three derived factors, but the direct-RAG denominator is larger because it recovered more true positives.

F1 summarizes the precision-recall balance. For RAG without HyDE, micro F1 is 0.745098. For RAG with HyDE, micro F1 is 0.723404. Since direct RAG has both higher precision and higher recall in the latest deterministic run, its higher F1 is expected rather than surprising. The macro F1 values preserve the same ordering, 0.731241 for direct RAG and 0.700821 for HyDE, after averaging per-query behavior.

For the recorded reference set, RAG without HyDE has the higher extraction scores. Both modes also produce 23 false positives, which indicates over-extraction relative to that reference. The evaluation does not isolate whether these errors arise from retrieval, synthesis, canonical matching, or limits in the derived reference set.

4.4.4. Runtime and performance

The complete chained execution in an NVIDIA 5090 32GB lasted 34 hours, 32 minutes, and 35 seconds. The generation, RAGAS, extraction, and robustness stages completed after 32 hours, 47 minutes, and 46 seconds. Retrieval preparation then ran for 1 hour, 44 minutes, and 42 seconds. Saving each phase's output allows the completed results to be analysed without repeating every evaluator call. Table 6 summarizes the runtime durations for various evaluation tasks.

Table 6. Runtime summary for the complete chained evaluation.

Measurement	Observed value
Complete chained execution	34 h 32 min 35 s
Generation, RAGAS, extraction, and robustness stages	32 h 47 min 46 s
Retrieval preparation	1 h 44 min 42 s
Phase 3 generation, all runs	12 h 19 min 10 s
Phase 4 RAGAS evaluation, all runs	20 h 26 min 56 s
Mean generation latency, LLM only	30.15 s/request
Mean generation latency, RAG without HyDE	126.74 s/request
Mean generation latency, RAG with HyDE	138.77 s/request

Phase 3 (Answer Generation) across the deterministic run and five robustness executions consumed approximately 12 hours, 19 minutes, and 10 seconds. Mean generation latency across 150 requests per mode was 30.15 seconds for LLM only, 126.74 seconds for RAG without HyDE, and 138.77 seconds for RAG with HyDE. Phase 4 (RAGAS Evaluation) consumed approximately 20 hours, 26 minutes, and 56 seconds, making it the dominant execution bottleneck. Aggregated by mode, Phase 4 consumed 1 hour, 5 minutes, and 52 seconds for LLM only, 9 hours, 48 minutes, and 46 seconds for RAG without HyDE, and 9 hours, 32 minutes, and 19 seconds for RAG with HyDE.

The latency values clarify the cost of retrieval augmentation. The LLM-only path averages about 30 seconds per request because it sends the prompt directly to the model. RAG without HyDE averages about 127 seconds because it includes retrieval, context preparation, and answer synthesis. RAG with HyDE averages about 139 seconds because it adds hypothetical-document generation and additional retrieval work. Evidence grounding and query expansion therefore have measurable cost in the recorded local environment.

The runtime profile has architectural consequences. A locally controlled Ollama deployment and CPU embedding setup support repeatable execution, but they constrain throughput. RAGAS is the main bottleneck. If scoring is expensive, evaluations may be run less often or on smaller samples. Larger question sets or repeated evaluator-calibration runs would require runtime optimization and safe parallelization.

The logs repeatedly warn that the evaluator returned one generation instead of the requested three. RAGAS scoring completed and produced the reported metric files, but this evaluator behavior and runtime configuration limit their interpretation. The logs also contain a Python 3.14 compatibility warning for Pydantic V1 functionality; this is a runtime caveat rather than a central scientific finding.

4.4.5. Robustness

The generated RAG answers and contexts are identical across robustness seeds except for request identifiers. Their RAGAS and extraction scores therefore have zero variance. This is a consequence of the current execution path and service defaults, not evidence of statistical robustness, because the RAG API did not receive the request-level stochastic controls passed to the LLM-only baseline.

The five-seed robustness experiment exercises stochastic variation for the LLM-only baseline, whose answer relevancy mean is 0.539965 with sample standard deviation 0.068130 across five samples. The standard deviation describes how much the five scores vary around their mean. For the RAG modes, zero variance cannot be interpreted in the same way because the RAG API did not receive equivalent request-level controls.

Chapter 5| Discussion

The primary evaluation supports a dimension-specific conclusion. LLM only has the highest answer relevancy but cannot be assessed for grounding because it has no retrieved contexts. RAG with HyDE has higher context precision without reference and slightly higher answer relevancy than RAG without HyDE. RAG without HyDE has higher faithfulness and extraction scores.

HyDE is neither uniformly harmful nor uniformly beneficial. Relative to RAG without HyDE, it slightly increases answer relevancy and context precision without reference, but it has lower faithfulness and extraction scores. The retrieval evaluation also shows opposing results: HyDE has higher MRR and Recall@5, while direct retrieval has higher Precision@5 and nDCG@5. Retrieval and generation should therefore be compared as separate phases rather than merged into a claim of universal superiority.

The phase-separated evaluation makes these differences visible. An improvement in one retrieval metric does not guarantee higher faithfulness or extraction scores, and an improvement in a generation metric can occur while structured extraction worsens.

The human curation process supports corpus preparation but does not establish clinical validation. Reviewed Label Studio records are exported, bibliographic metadata are enriched, clean records are separated from rejected records, and ingestion-ready publications are sent to the biomedical RAG API. The available material does not establish inter-annotator agreement, adjudication, prospective validation, or clinical outcome testing. A clinically stronger study would require independent expert review, disagreement adjudication, clearer population and outcome definitions, and validation against clinical standards.

The main engineering strength is separation of concerns. The FastAPI layer is distinct from use cases, domain models, ports, and infrastructure adapters. The scheduler calls document and ingestion endpoints instead of writing to the vector database. The evaluation subsystem calls public APIs and stores phase outputs. At larger scale, durable ingestion state, vector-store lifecycle management, corpus versioning, model versioning, and larger evaluation datasets remain important.

The principal RAGAS error pattern is a support tradeoff. HyDE appears to focus the context-support proxy, but the resulting answers are less faithful to the retrieved contexts under the evaluator. This suggests that higher context precision without reference should not be read as sufficient evidence of grounded biomedical correctness.

The principal extraction error pattern is over-extraction. As summarized in Table 5 both RAG configurations produce the same false-positive and false-negative counts, while RAG without HyDE recovers more true positives. These results cover only the 13 questions with derived reference factors; the remaining 12 are outside the extraction denominator.

5.1. Critical discussion

The separation of retrieval from answer generation follows biomedical evaluation settings such as BioASQ and TREC Precision Medicine, where evidence ranking is measured independently of downstream answers [16],[17]. The different directions of the retrieval, RAGAS, and extraction results support this separation. Direct numerical comparison with those studies is not appropriate because their corpora, questions, and reference data differ.

The disagreement between context precision and faithfulness also supports the fine-grained approach used by RAGAS, ARES, and RAGChecker [29],[30],[31]. A single aggregate score would hide the higher HyDE context-precision value and the higher direct-RAG faithfulness value. As reported in medical RAG research, these automated measures diagnose system behavior but do not establish autonomous clinical use [25].

The reviewed annotation records support extraction analysis, but the available process is less complete than corpus-construction methods that report independent annotation, agreement, and adjudication [32],[33],[34]. This difference limits comparison with established biomedical reference datasets and reinforces the need for conservative interpretation.

5.2. Limitations

The first limitation is evaluation size. The question set contains 25 questions. This is appropriate for examining the research prototype but does not support broad statistical or clinical claims.

The second limitation is reference construction. Extraction factors are derived from reviewed Label Studio records, confidence policy, cited DOI alignment, and canonical factor matching. This reference set does not constitute an adjudicated clinical gold standard. Retrieval qrels are also bounded by their recorded state, annotation quality, and the absence of multi-annotator adjudication.

The third limitation is stochastic-control validity. Robustness seed and temperature controls currently propagate to the LLM-only direct Ollama call, but not equivalently to the RAG API path. Zero variance in RAG robustness should therefore not be interpreted as stochastic robustness. Future robustness experiments should expose explicit request-level seed and temperature controls for RAG synthesis and HyDE generation and should define HyDE cache controls before making stochastic robustness claims.

The fourth limitation is evaluator behavior. RAGAS scoring depends on the evaluator model and produced repeated warnings that one generation was returned where three were requested. The recorded metrics remain useful for comparison, but this limitation affects confidence in evaluator stability. Future work should review evaluator configuration and calibrate automated judgements against expert review.

A further limitation is incomplete control of external state. The run directory, configuration snapshot, phase outputs, and log-derived timings are recorded, but complete vector-store snapshots and Ollama model digests are not. Stronger scientific comparisons would require these identifiers as run metadata.

5.3. Research questions and main findings

The evaluation demonstrates a phase-separated biomedical RAG evaluation in which retrieval, generation, extraction, and runtime are reported independently. The results are not uniform across these measured functions.

- Retrieval: HyDE has higher MRR and Recall@5, while direct retrieval has higher Precision@5 and nDCG@5.
- Generation: RAG without HyDE has higher faithfulness than HyDE-enabled RAG, while HyDE has higher context precision without reference.
- Extraction: RAG without HyDE has a micro F1 of 0.745 compared with 0.723 for HyDE on the 13 covered questions. Both configurations produce 23 false positives.

- Runtime: RAGAS scoring is the main bottleneck, consuming more than 20 hours of the 34-hour chained execution.

The central contribution is a biomedical RAG platform and evaluation process that make retrieval, grounding, extraction, and runtime tradeoffs visible. The results do not establish clinical validity or readiness for patient care.

5.4. Future work

Future work should first strengthen stochastic controls. The RAG API should expose request-level seed and temperature controls for synthesis, and HyDE generation should expose equivalent controls. HyDE cache behavior should also be configurable during robustness experiments, so zero variance can be distinguished from true stochastic stability.

This work should include an explicit robustness contract for every generation path. The evaluation CLI, RAG API, HyDE generator, and synthesis service should agree on how seed, temperature, cache behavior, and model options are passed and recorded. Without that contract, future robustness tables may again mix stochastic and deterministic paths under the same experimental label.

The second priority is retrieval annotation governance. The current retrieval evaluation supports the reported ranking metrics, but future evaluations should use multi-annotator qrels adjudication, preserve adjudication status, and document annotation governance.

The qrels process should also be versioned. Each retrieval evaluation should record the query set, corpus snapshot, candidate pool, annotation template, completed qrels file, adjudication policy, per-query metrics, and summary metrics together. This would identify the corpus state to which each retrieval result belongs.

The third priority is larger and more clinically reviewed extraction coverage. More questions should have derived reference factors, and evaluator calibration should compare RAGAS and extraction outputs against expert judgment. This would make it possible to estimate how well automated scores align with biomedical review rather than relying on metric values alone.

Expanded extraction coverage should preserve the distinction between reproducible derived reference factors and clinical gold standards. A practical next step is to increase the number of questions with derived factors and then introduce expert adjudication for a smaller validation subset. This would allow two separate questions to be studied: whether the system agrees with factors derived from the reviewed records, and whether those factors align with expert biomedical judgment.

The fourth priority is experiment provenance. Future runs should record corpus version identifiers, Qdrant collection snapshots or snapshot metadata, Ollama model digests, evaluator model and runtime digests, and evaluation code versions. These records would reduce ambiguity when comparing results across changing corpora and local inference environments.

Provenance should be a standard evaluation output rather than informal context. A future run directory should identify the indexed corpus, vector collection, embedding model, Ollama model digest, code revision, and annotation files used for each metric. Without this information, changes in results can be difficult to attribute.

The fifth priority is runtime optimization. The June execution shows that RAGAS evaluation dominates runtime. Larger evaluations will require batching review, safe parallel evaluation where possible, evaluator configuration review, and possibly smaller calibrated screening runs before the complete evaluation.

Runtime optimization should preserve auditability. Parallel execution is useful only if phase outputs remain deterministic where expected, logs remain attributable to specific requests, and failed evaluator calls can be diagnosed. The correct target is faster evaluation with preserved traceability.

Additional future work remains valuable but secondary to evaluation validity: biomedical embedding comparisons, conditional HyDE activation, retrieval explanations, durable ingestion queues, and clinical workflow studies. These extensions should be judged by faithfulness, extraction quality, retrieval metrics, latency, failure behavior, and expert review rather than by anecdotal answer quality.

5.5. Closing remarks

The work produced a biomedical RAG platform rather than a standalone model demonstration. The implemented system consists of a modular FastAPI biomedical RAG service, a PubMed scheduler, a phase-based evaluation subsystem, local Ollama model serving, and human-reviewed literature processing. The biomedical RAG service performs asynchronous ingestion, section-aware chunking, sentence-transformer embedding, Qdrant vector indexing, retrieval, optional HyDE expansion, reranking, and cited answer synthesis. Thrombosis risk-factor identification was the use case used to demonstrate and evaluate this general biomedical literature architecture.

The main contribution is the integration of these components through explicit boundaries. API endpoints remain separated from use cases, domain objects, ports, and infrastructure adapters. Qdrant, Ollama, PubMed, annotation exports, and evaluation clients remain external integrations rather than hidden domain logic. The project therefore contributes an engineering research platform with recorded evaluation files, not a claim of clinical superiority.

Evaluation design is also part of the system architecture. Beyond exposing an answer endpoint, the platform preserves generation, RAGAS, extraction, robustness, retrieval, and log files. This separates answer relevance from grounding, extraction quality from retrieval ranking, and repeatable execution from stochastic robustness. A single attractive metric cannot therefore hide all other failure modes.

The work demonstrates a modular, locally deployable biomedical RAG platform with recorded evaluation outputs and human curation support. Thrombosis risk-factor identification provides the evaluation use case, not the platform's only possible application. The results favour RAG without HyDE for faithfulness and extraction, show higher HyDE answer relevancy and context precision without reference, and show a separate retrieval tradeoff in which HyDE has higher MRR and Recall@5 while direct retrieval has higher Precision@5 and nDCG@5. These findings do not establish clinical readiness.

References

[1] Sayers E, A general introduction to the E-utilities, Entrez Programming Utilities Help, National Center for Biotechnology Information, updated Nov. 17, 2022. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK25497/

[2] Fiorini N, Canese K, Starchenko G, Kireev E, Kim W, Miller V, Osipov M, Kholodov M, Ismagilov R, Mohan S, Ostell J, Lu Z. Best Match: New relevance search for PubMed. PLoS Biol. 2018 Aug 28;16(8):e2005343. https://doi.org/10.1371/journal.pbio.2005343

[3] Lewis P, erez E, Piktus A, Pertoni F, Karpukhin V, Goyal N, Kuttler H Lewis M, Yih W, Rocktaschel T, Riedel S, Keila D, Retrieval-augmented generation for knowledge-intensive NLP tasks, arXiv:2005.11401, 2020, https://doi.org/10.48550/arXiv.2005.11401

[4] Monie DD, DeLoughery EP. Pathogenesis of thrombosis: cellular and pharmacogenetic contributions. Cardiovasc Diagn Ther. 2017 Dec;7(Suppl 3):S291-S298. https://doi.org/10.21037/cdt.2017.09.11

[5] Raskob GE, Angchaisuksiri P, Blanco AN, Buller H, Gallus A, Hunt BJ, Hylek EM, Kakkar A, Konstantinides SV, McCumber M, Ozaki Y, Wendelboe A, Weitz JI; ISTH Steering Committee for World Thrombosis Day. Thrombosis: a major contributor to global disease burden. Arterioscler Thromb Vasc Biol. 2014 Nov;34(11):2363-71. https://doi.org/10.1161/ATVBAHA.114.304488

[6] Wendelboe A, Weitz JI. Global Health Burden of Venous Thromboembolism. Arterioscler Thromb Vasc Biol. 2024 May;44(5):1007-1011. https://doi.org/10.1161/ATVBAHA.124.320151

[7] Grosse SD, Nelson RE, Nyarko KA, Richardson LC, Raskob GE. The economic burden of incident venous thromboembolism in the United States: A review of estimated attributable healthcare costs. Thromb Res. 2016 Jan;137:3-10. https://doi.org/10.1016/j.thromres.2015.11.033

[8] Valerio L, Christodoulou KC, Abele C, Ten Cate V, Keller K, Kucher N, Middeldorp S, Prandoni P, Righini M, Konstantinides SV, Barco S. Postthrombotic syndrome: risk after deep vein thrombosis and estimates of its incidence and prevalence in Europe. J Thromb Haemost. 2026 May;24(5):1690-1699. https://doi.org/10.1016/j.jtha.2025.12.017

[9] Khorana AA, Mackman N, Falanga A, Pabinger I, Noble S, Ageno W, Moik F, Lee AYY. Cancer-associated venous thromboembolism. Nat Rev Dis Primers. 2022 Feb 17;8(1):11. https://doi.org/10.1038/s41572-022-00336-y

[10] Dimakakos E, Gomatou G, Catalano M, Olinic DM, Spyropoulos AC, Falanga A, Maraveyas A, Liew A, Schulman S, Belch J, Gerotziafas G, Marschang P, Cosmi B, Spaak J, Syrigos K; CANCER-COVID-19 THROMBOSIS COLLABORATIVE GROUP, endorsed by VAS-European Independent Foundation in Angiology/Vascular Medicine, UEMS Vascular Medicine/Angiology and European Society of Vascular Medicine, and supported by the Balkan Working Group for Prevention and Treatment of Venous Thromboembolism and Hellenic Association of Lung Cancer. Thromboembolic Disease in Patients With Cancer and COVID-19: Risk Factors, Prevention and Practical Thromboprophylaxis Recommendations-State-of-the-Art. Anticancer Research, 42(7):3261-3274, 2022 https://doi.org/10.21873/anticanres.15815

[11] Clapham RE, Speed V, Cox C, Patel JP, Roberts LN. Identifying risk factors for venous thromboembolism in medical inpatients: a systematic review and meta-analysis. Res Pract Thromb Haemost. 2026 Apr 30;10(4):106625. https://doi.org/10.1016/j.rpth.2026.106625

[12] Zuo T, Yu J, Yin X, Wu H, Mao S. Incidence and risk factors of deep vein thrombosis in critically ill patients: a systematic review and meta-analysis. Thromb J. 2026 May 11;24(1):63. https://doi.org/10.1186/s12959-026-00870-9

[13] Ren Y, Liu R. Prevalence and risk factors of deep vein thrombosis in elderly: a systematic review and meta-analysis. Open Med (Wars). 2026 Jan 21;21(1):20251333. https://doi.org/10.1515/med-2025-1333

[14] Reyes NL, Abe K. Deep Vein Thrombosis and Pulmonary Embolism. 2025 Apr 23. In: Halsey ES, Angelo KM, Barnett ED, et al., editors. CDC Yellow Book, 2026 edition: Health Information for International Travel [Internet]. Atlanta (GA): Centers for Disease Control and Prevention (US); 2025 Apr 23. Available from: https://www.ncbi.nlm.nih.gov/books/NBK620950/

[15] Gao L, Ma X, Lin J, Callan J. Precise zero-shot dense retrieval without relevance labels, Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1762–1777, 2023, https://doi.org/10.18653/v1/2023.acl-long.99

[16] Tsatsaronis G, Balikas G, Malakasiotis P, Partalas I, Zschunke M, Alvers MR, Weissenborn D, Krithara A, Petridis S, Polychronopoulos D, Almirantis Y, Pavlopoulos J, Baskiotis N, Gallinari P, Artiéres T, Ngomo AC, Heino N, Gaussier E, Barrio-Alvers L, Schroeder M, Androutsopoulos I, Paliouras G. An overview of the BIOASQ large-scale biomedical semantic indexing and question answering competition. BMC Bioinformatics. 2015 Apr 30;16:138. https://doi.org/10.1186/s12859-015-0564-6

[17] Roberts K, Demner-Fushman D, Voorhees EM, Bedrick S, Hersh WR. Overview of the TREC 2020 Precision Medicine Track. Text Retr Conf. 2020 Nov; https://trec.nist.gov/pubs/trec29/papers/OVERVIEW.PM.pdf

[18] Jin Q, Kim W, Chen Q, Comeau DC, Yeganova L, Wilbur WJ, Lu Z. MedCPT: Contrastive Pre-trained Transformers with large-scale PubMed search logs for zero-shot biomedical information retrieval. Bioinformatics. 2023 Nov 1;39(11):btad651. https://doi.org/10.1093/bioinformatics/btad651

[19] Lee J, Yoon W, Kim S, Kim D, Kim S, So CH, Kang J. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. Bioinformatics. 2020 Feb 15;36(4):1234-1240. https://doi.org/10.1093/bioinformatics/btz682

[20] Reimers N, Gurevych I. “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” arXiv:1908.10084, 2019. https://doi.org/10.48550/arXiv.1908.10084

[21] Lozano A, Fleming SL, Chiang CC, Shah N. Clinfo.ai: An Open-Source Retrieval-Augmented Large Language Model System for Answering Medical Questions using Scientific Literature. Pac Symp Biocomput. 2024;29:8-23.

[22] Zakka C, Shad R, Chaurasia A, Dalal AR, Kim JL, Moor M, Fong R, Phillips C, Alexander K, Ashley E, Boyd J, Boyd K, Hirsch K, Langlotz C, Lee R, Melia J, Nelson J, Sallam K, Tullis S, Vogelsong MA, Cunningham JP, Hiesinger W. Almanac - Retrieval-Augmented Language Models for Clinical Medicine. NEJM AI. 2024 Feb;1(2):10.1056/aioa2300068. https://doi.org/10.1056/aioa2300068

[23] Xiong G, Jin Q, Lu Z, Zhang A, Benchmarking retrieval-augmented generation for medicine, Findings of the Association for Computational Linguistics: ACL 2024, pp. 6233–6251, 2024. https://doi.org/10.18653/v1/2024.findings-acl.372

[24] Amugongo LM, Mascheroni P, Brooks S, Doering S, Seidel J. Retrieval augmented generation for large language models in healthcare: A systematic review. PLOS Digit Health. 2025 Jun 11;4(6):e0000877. https://doi.org/10.1371/journal.pdig.0000877

[25] Malik S, Kharel H, Dahiya DS, Ali H, Blaney H, Singh A, Dhar J, Perisetti A, Facciorusso A, Chandan S, Mohan BP. Assessing ChatGPT4 with and without retrieval-augmented generation in anticoagulation management for gastrointestinal procedures. Ann Gastroenterol. 2024 Sep-Oct;37(5):514-526. https://doi.org/10.20524/aog.2024.0907

[26] Jarvelin K, Kekalainen J, Cumulated gain-based evaluation of IR techniques, ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002, https://doi.org/10.1145/582415.582418

[27] Thakur N, Reimers N, Ruckle A, Srivastava A, Gurevych I. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. arXiv:2104.08663, 2021. https://doi.org/10.48550/arXiv.2104.08663

[28] Jin Q, Dhingra B, Liu Z, Cohen W, Lu X. PubMedQA: A dataset for biomedical research question answering. Proc. EMNLP-IJCNLP, pp. 2567–2577, 2019. https://doi.org/10.18653/v1/D19-1259

[29] Es S, J. James J, Espinosa-Anke L, Schockaert S. RAGAs: Automated evaluation of retrieval augmented generation, Proc. EACL: System Demonstrations, pp. 150–158, 2024, 2024. https://doi.org/10.18653/v1/2024.eacl-demo.16

[30] Saad-Falcon J, Khattab O, Potts C, Zaharia M. ARES: An automated evaluation framework for retrieval-augmented generation systems. Proc. NAACL-HLT, pp. 338–354, 2024. https://doi.org/10.18653/v1/2024.naacl-long.20

[31] Ru D, Qiu L, J=Hu X, Zhang T, Shi P, Chang S, Jiayang C, Wang C, Sun S, Li H, Zhang Z, Wang B, Jiang J, He T, Wang Z, Liu P, Zhang Y, Zhang Z. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. Proc. NeurIPS Datasets and Benchmarks, pp. 21999-22027, 2024. https://doi.org/10.52202/079017-0692

[32] Deleger L, Li Q, Lingren T, Kaiser M, Molnar K, Stoutenborough L, Kouril M, Marsolo K, Solti I. Building gold standard corpora for medical natural language processing tasks. AMIA Annu Symp Proc. 2012; 2012:144-53

[33] Stubbs A, Kotfila C, Xu H, Uzuner Ö. Identifying risk factors for heart disease over time: Overview of 2014 i2b2/UTHealth shared task Track 2. J Biomed Inform. 2015 Dec;58 Suppl(Suppl):S67-S77. https://doi.org/10.1016/j.jbi.2015.07.001

[34] Stubbs A, Uzuner Ö. Annotating risk factors for heart disease in clinical narratives for diabetic patients. J Biomed Inform. 2015 Dec;58 Suppl(Suppl):S78-S91. https://doi.org/10.1016/j.jbi.2015.05.009

[35] Névéol A, Islamaj Doğan R, Lu Z. Semi-automatic semantic annotation of PubMed queries: a study on quality, efficiency, satisfaction. J Biomed Inform. 2011 Apr;44(2):310-8 https://doi.org/10.1016/j.jbi.2010.11.001

[36] Vaid A, Duong SQ, Lampert J, Kovatch P, Freeman R, Argulian E, Croft L, Lerakis S, Goldman M, Khera R, Nadkarni GN. Local large language models for privacy-preserving accelerated review of historic echocardiogram reports. J Am Med Inform Assoc. 2024 Sep 1;31(9):2097-2102. https://doi.org/10.1093/jamia/ocae085

[37] Wiest IC, Ferber D, Zhu J, van Treeck M, Meyer SK, Juglan R, Carrero ZI, Paech D, Kleesiek J, Ebert MP, Truhn D, Kather JN. Privacy-preserving large language models for structured medical information retrieval. NPJ Digit Med. 2024 Sep 20;7(1):257. https://doi.org/10.1038/s41746-024-01233-2

[38] Thirunavukarasu AJ, Ting DSJ, Elangovan K, Gutierrez L, Tan TF, Ting DSW. Large language models in medicine. Nat Med. 2023 Aug;29(8):1930-1940. https://doi.org/10.1038/s41591-023-02448-8

[39] Riedemann L, Labonne M, Gilbert S. The path forward for large language models in medicine is open. NPJ Digit Med. 2024 Nov 27;7(1):339. https://doi.org/10.1038/s41746-024-01344-w

[40] Dubey A. et al., The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783 Accessed: Jun. 1, 2026.

[41] Carlini N, Tramer F, Wallace E, Jagielski M, Herbert-Voss A, Lee K, Roberts A, Brown T, Song D, Erlingsson U, Oprea A, Reaffel C. Extracting training data from large language models. Proc. 30th USENIX Security Symposium, pp. 2633–2650. 2021. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting Accessed: Jun. 1, 2026.

[42] Martin RC, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA, USA: Pearson, 2018.

[43] Cockburn A. Hexagonal architecture: The original 2005 article. 2005. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/

[44] Qdrant, Collections. Qdrant Documentation. [Online]. Available: https://qdrant.tech/documentation/manage-data/collections/ . Accessed: May 23, 2026.

[45] Ollama, "Introduction," Ollama API Documentation. [Online]. Available: https://docs.ollama.com/api/introduction Accessed: May 23, 2026.

[46] Tkachenko M, Malyuk M, Holmanyuk A, Liubimov N. Label Studio: Data labeling software, 2020–2025. [Online]. Available: https://github.com/HumanSignal/label-studio

[47] Cormack GV, Clarke CLA, Buettcher S. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods, Proc. SIGIR, pp. 758–759, 2009. doi: 10.1145/1571941.1572114.

[48] Docker Inc., Docker Compose, *Docker Documentation*. [Online]. Available: https://docs.docker.com/compose/. Accessed: Jun. 1, 2026.

[49] O. Ben-Kiki, C. Evans, and I. Net, "YAML Ain't Markup Language (YAML) version 1.2.2," YAML Language Development Team, Oct. 1, 2021. [Online]. Available: https://yaml.org/spec/1.2.2/

ANNEX 1. Repository Structure Overview

This appendix summarizes the repository structure. The repository is organized as a monorepo containing the thesis source, service implementation, evaluation framework, local model infrastructure, and curation tooling. This organization links the architectural description to the implementation and the reported results to the generated evaluation files.

```
rag/
  biomed_knowledge_platform/     # FastAPI biomedical RAG service
  scheduler_pubmed/              # PubMed acquisition scheduler
  evaluation_metrics/            # Phase-based evaluation pipeline
  llm_server/                    # Ollama local model-serving stack
  labelstudio-tools/             # Label Studio curation utilities
  docs/thesis/latex/             # Thesis source and generated PDF
  docs/thesis/evidence/          # Source and result registry
```

The biomedical RAG service follows a Clean Architecture and ports-and-adapters structure. Transport concerns live in the API layer, orchestration lives in use cases, domain models and ports remain framework-independent, and infrastructure integrations are implemented as adapters. This separation allows retrieval, ingestion, generation, and audit behavior to be considered independently.

```
biomed_platform/
  api/        # routers, endpoint schemas, error handlers
  core/       # domains, use cases, ports, retrieval services
  adapters/   # Qdrant, Ollama, PubMed clients
  db/         # SQLAlchemy engine, models, Alembic integration
  audit/      # PostgreSQL-backed audit service
  common/     # settings, logging, middleware, utilities
```

The scheduler owns PubMed acquisition and downstream ingestion coordination, while the evaluation subsystem runs the evaluation phases and writes their output files. Neither subsystem directly changes the internals of the other. This separation avoids embedding evaluation-specific logic into the online service

ANNEX 2. Configuration Architecture

Configuration is deliberately externalized. The platform uses YAML files [49] for service configuration and evaluation configuration, while Label Studio tooling uses environment variables for curation-specific secrets and runtime paths.

Retrieval Configuration

The RAG configuration records the embedding provider, device, chunking strategy, vector backend, and default retrieval depth. These values affect both system behavior and evaluation interpretation because they define the embedding space, chunk granularity, and candidate pool size used before reranking and answer synthesis.

```
rag:
    embedding:
        provider: "sentence-transformers/all-MiniLM-L6-v2"
        device: "cpu"
        normalize_embeddings: false
    chunking:
        chunk_size: 1200
        overlap: 150
    vector_index:
        backend: "qdrant"
    retrieval:
        top_k: 10
```

Vector Store Configuration

The Qdrant configuration records the local endpoint, timeout, collection prefix, and distance metric. Collection names are derived from the configured prefix and embedding model identifier, which supports model-specific vector collections while preserving a stable service contract.

```
qdrant:
    url: "http://localhost:6333"
    timeout_seconds: 30.0
    collection:
        name_prefix: "docs"
        distance: "COSINE"
```

LLM Configuration

The LLM configuration records the Ollama endpoint, generation model, HyDE model, prompt and context bounds, concurrency, retry count, and timeout. These values are important for reproducibility because the same retrieval contexts can produce different operational behavior when model identity, timeout, concurrency, or generation options change.

```
ollama_base_url: "http://192.168.2.3:11434"
generator_model_id: "llama3.1:8b-instruct-q4_K_M"
```

```
hyde_enabled: false
ask_max_context_chars: 60000
ask_max_chunks_candidate: 30
ask_max_chunks_final: 6
llm_max_concurrency: 2
```

Evaluation Configuration

The evaluation configuration defines the RAG API endpoint, HyDE header, model settings, deterministic and robustness seeds, query file, RAGAS embedding model, RAGAS embedding device, extraction policy, and k values. This file is copied into each run directory as a configuration snapshot, linking the evaluation outputs to the settings that produced them. The generation evaluation uses 25 questions, primary seed 42, primary temperature 0.0, and robustness seeds 11, 22, 33, 44, and 55 at configured temperature 0.2. The retrieval evaluation uses the same query set but evaluates ranked evidence with completed qrels-based metrics.

```
ragas:
    embeddings_device: cuda
paper:
    primary_seed: 42
    primary_temperature: 0.0
    robustness_temperature: 0.2
    robustness_seeds: [11, 22, 33, 44, 55]
metrics:
    k_values: [5, 10, 20]
extraction:
    enabled: true
    tasks_clean_json: evaluation_metrics/tasks_clean.json
```

The example in this appendix uses a verified CUDA-oriented evaluation profile. Commit a1faf166 introduced the configuration used on the RTX 5090 workstation by moving the evaluation-side RAGAS embeddings from cpu to cuda. The change applies to eval.yaml, eval.fast.yaml, and eval.medium.yaml, and also aligns dependent service endpoints with localhost-based execution. This documents how the evaluation subsystem may be provisioned on a single CUDA-capable workstation; it does not alter the meaning of the reported metrics.

ANNEX 3. Deployment and Architecture

Deployment is local-first and Compose-based. Docker Compose runs the local dependencies; the implementation includes no Helm or Kubernetes deployment files. This setup supports repeatable local execution, but it is not production-grade orchestration.

The biomedical RAG stack defines Qdrant, PostgreSQL, and the API service as separate Compose services. Qdrant persists vectors in a named volume, PostgreSQL persists relational state in a named volume, and the API mounts configuration files read-only. This separation mirrors the application architecture: vector retrieval, relational state, and application behavior are independently configurable infrastructure concerns.

```
services:
    qdrant:
        image: qdrant/qdrant:v1.16.3
        ports: ["6333:6333", "6334:6334"]
    postgres:
        image: postgres:17.7
        ports: ["5432:5432"]
    api:
        build: .
        ports: ["8000:8000"]
        volumes:
        - ./configs:/service/configs:ro
```

The LLM stack is separate from the biomedical RAG service. It contains Ollama for model serving, Open WebUI for local interaction, and Traefik for local routing. This separation lets the RAG service treat LLM inference as an HTTP dependency while keeping model-hosting lifecycle concerns outside the application code.

```
services:
    ollama:
        build: ./ollama
        volumes:
            - ollama_models:/root/.ollama
    open-webui:
        image: ghcr.io/open-webui/open-webui:0.6.43
        environment:
        - OLLAMA_BASE_URL=http://ollama:11434
```

ANNEX 4. Evaluation Pipeline Details

The evaluation subsystem is implemented as a command-line pipeline with explicit phases. It writes the output of each phase to files for later analysis. The following subsections describe the generation and extraction files and the independent retrieval files.

Evaluation Pipeline Implementation

The evaluation subsystem follows the phase-separated architecture defined in Table 4.1 (Chapter 4). Each phase performs a specific operation and writes its output before the next phase begins. This design allows each stage of an evaluation run to be examined independently.

Files pass sequentially through the pipeline. The retrieval evaluation in Phases 1 and 2 measures evidence ranking, while the generation evaluation in Phases 3, 4, and 6 measures the synthesized answers and extracted risk factors. This section documents the directory structure and file names produced by these phases.

Generation Evaluation Files

The reported generation, RAGAS, extraction, and robustness outputs are stored in the archived June 7 run directory 20260605_123942_nogit. The deterministic primary run stores one answer file per mode, RAGAS outputs for each answer file, extraction outputs for both RAG modes, and aggregate summary files. Robustness outputs are stored under five seed-specific directories.

```
20260605_123942_nogit/
  config_snapshot.json
  paper_benchmark_summary.json
  primary_deterministic/
    phase3_answers_llm_only.jsonl
    phase3_answers_rag_no_hyde.jsonl
    phase3_answers_rag_hyde.jsonl
    phase4_ragas_phase3_answers_*.jsonl
    phase6_extraction_phase3_answers_rag_no_hyde_summary.csv
    phase6_extraction_phase3_answers_rag_hyde_summary.csv
  robustness/
    seed_11/
    seed_22/
    seed_33/
    seed_44/
    seed_55/
```

Generation evaluation records contexts extracted from the ask response rather than from a separate search call. This matters because faithfulness must be evaluated against the evidence actually supplied to the synthesis stage. Extraction evaluation compares predicted factors with a derived reference set and skips queries that have no derived reference factors under the configured policy.

Retrieval Evaluation Files

The reported retrieval evaluation is stored in the archived June 7 run directory . It contains ranked direct retrieval records, ranked HyDE retrieval records, final context records, a pooled candidate set, a completed annotation file, qrels.tsv, per-query metrics, and aggregate retrieval summaries.

20260606_212735_nogit/
config_snapshot.json
retrieval_direct.jsonl
retrieval_hyde.jsonl
retrieval_final_context.jsonl
pooled_candidates.jsonl
qrels_annotation_template.csv
qrels_annotation_completed.csv
qrels.tsv
retrieval_metrics_summary.json
retrieval_metrics_per_query.csv
retrieval_metrics_comparison.tex

The retrieval evaluation contains 25 queries, 250 direct retrieval rows, 250 HyDE retrieval rows, 300 final context rows, 321 pooled candidates, completed qrels files, retrieval metric summaries, and retrieval comparison tables. These files support the retrieval results reported in Chapter 7.

This separation is essential because RAGAS context precision without reference remains an answer-support proxy, extraction metrics evaluate risk-factor recovery against a derived reference set, and qrels-based retrieval metrics evaluate ranked documents against query-specific relevance judgments.

ANNEX 5. Configuration Examples

This section presents small configuration excerpts that identify the runtime shape of the platform. The full configuration files contain the complete execution settings.

RAG Service Snippet

```
embedding:
  provider: "sentence-transformers/all-MiniLM-L6-v2"
  device: "cpu"
chunking:
  chunk_size: 1200
  overlap: 150
retrieval:
  top_k: 10
```

Scheduler Snippet

```
api:
  base_url: http://localhost:8000
  documents_get: /v1/documents/
  documents_post_batch: /v1/documents/fetch/batch
  ingest_jobs_get: /v1/ingest/jobs/
```

Evaluation Snippet

```
ask:
  hyde_header_name: X-HyDE-Enabled
paths:
  queries_jsonl: evaluation_metrics/queries/queries.jsonl
extraction:
  tasks_clean_json: evaluation_metrics/tasks_clean.json
```

Compose Snippet

```
volumes:
  qdrant_storage:
  postgres_data:
services:
  qdrant:
    image: qdrant/qdrant:v1.16.3
  postgres:
    image: postgres:17.7
```

Reproducibility Notes

The saved configuration and evaluation files support repeated analysis of the reported experiments. The Makefile builds the thesis with two XeLaTeX passes, and the evaluation CLI writes timestamped run directories

with configuration snapshots. The evidence index identifies the run directories used for the reported generation and retrieval results.

Local LLM behavior remains a reproducibility constraint. The service uses a locally reachable Ollama endpoint and a quantized Llama model. Even with deterministic seeds in the evaluation configuration, exact rerun identity can still depend on model files, host hardware, Ollama version, concurrency, timeout behavior, and service state. The saved output files record the completed run, but they do not guarantee byte-identical future generations.

Hardware and runtime dependencies also affect repeatability. The archived reported run used CPU-side evaluation embeddings, while the "cuda-validation-metrics" branch also documents a CUDA execution option for evaluation-side embeddings through commit a1faf166. Qdrant and PostgreSQL store local state in volumes, and the API uses in-memory ingestion queues and stores in the current composition. These settings should be recorded when comparing new runs with the reported run.

ANNEX 6. CUDA and APEX Preparation

CUDA execution for the evaluation subsystem should be prepared as an environment-validation procedure rather than as an informal host assumption. The required checks are straightforward. First, verify NVIDIA driver availability with nvidia-smi. Second, verify that the CUDA toolkit is available on the workstation and matches the intended runtime toolchain. Third, verify that the active PyTorch environment detects CUDA correctly before running the evaluation pipeline.

```
nvidia-smi

nvcc --version

python -c "import torch; print(torch.cuda.is_available())"
```

APEX should be treated conditionally. The evaluation dependency list does not declare NVIDIA APEX as mandatory, so APEX should only be installed or validated if a local execution profile requires it. When APEX is required, it must be built against the same CUDA toolkit and the same PyTorch environment that will execute the evaluation pipeline; mixing a different CUDA or PyTorch build is a common cause of import and runtime failures.

A short troubleshooting rule is sufficient for most failures. If CUDA is visible to the host but not to PyTorch, the problem is usually an environment or binary compatibility mismatch. If APEX imports fail, undefined symbol errors or extension build errors should be treated first as a signal that APEX was compiled against a different CUDA or PyTorch installation than the one currently active. In that situation, APEX should be rebuilt or reinstalled inside the execution environment intended for the evaluation run, rather than treating the failure as an evaluation anomaly.

ANNEX 7. Additional Technical Notes

Synthesis Prompt and Extraction Mapping

The following excerpts show the JSON schema prompt used for synthesis and the canonical mapping rules used for extraction.

The synthesis step enforces structured generation using a strict prompt schema:

```
You are a biomedical evidence grounded answer generator.
Hard rules
1. Output MUST be a single JSON object and nothing else.
2. No markdown, no prose, no commentary.
3. Use ONLY the provided chunk contexts, do not invent facts.
4. Cite ONLY allowed chunk_ids.
...
Required JSON schema
{
  "answer": {
    "summary": "string",
    "risk_factors": [
      {
        "rank": 1,
        "normalized_name": "string",
        "aliases": ["string"],
        "confidence": 0.5,
        "rationale": "string",
        "citations": ["chunk_id"]
      }
    ],
    "limitations": ["string"]
  },
  "citations": ["chunk_id"]
}
```

The Phase 6 extraction evaluation maps specific terminology back to its default canonical-factor dictionary. An excerpt of this mapping includes:

```
{
    "prior_vte": [
        "prior vte",
        "previous vte",
        "history of vte",
        "vte history",
        "venous thromboembolism history",
        "prior dvt",
        "history of dvt",
```

```
        "deep vein thrombosis history",
        "prior pulmonary embolism",
        "history of pulmonary embolism",
        "previous thrombosis",
      ],
      "age": [
        "advanced age",
        "older age",
        "elderly",
        "aging",
      ],
      "obesity": [
        "obesity",
        "obese",
        "body mass index",
        "bmi",
      ],
      "cancer": [
        "cancer",
        "malignancy",
        "neoplasm",
        "tumor",
      ],
      "immobility": [
        "immobility",
        "immobilization",
      ]
    }
```

Observed Label Studio Export Structure

The exported Label Studio tasks contain a top-level identifier and a data object holding the bibliographic payload. In the clean export, the data object includes at least pmid, title, abstract, doi, doi_url, source, source_id, source_url, year, journal, and authors. Additional export metadata includes timestamps, labeling status, project information, and the full list of annotations attached to the task.

The recorded export contains 1300 raw exported tasks, 1288 clean tasks, and 12 rejected tasks. All clean tasks are marked as labeled in the export metadata. Within the clean task file, 1277 tasks contain one annotation object and 11 tasks contain two annotation objects. The evaluation code currently reads the first annotation object for a task, so this export shape should be treated as an implementation assumption rather than as proof of adjudicated multi-reviewer consensus.

Within the observed annotations, the result array stores heterogeneous review items. These include span labels over abstract text, binary choices for relation to venous thrombosis and risk-factor reporting, confidence ratings, article-type choices, and clinical category selections. The annotation model combines inclusion

rationale, topical relevance, confidence, and coarse categorization, but the recorded export does not show identical annotation content for every Label Studio task.

For downstream ingestion, the pipeline applies a stricter validity schema than the raw export format. A task is ingestion-ready only when it contains non-empty source metadata, DOI, year, journal, authors, title, and abstract fields. Tasks failing these checks are written to a rejected-task report, and unresolved identifier cases are tracked separately. This means that the RAG handoff assumes metadata completeness, while annotation completeness remains a procedural expectation outside the current code-enforced contract.